

MN Golden Standard for Requirements Specification

MN Specification Process and WOW

Exported on 05/21/2026

Table of Contents

1	Chapter 1 - Requirements types and requirements documentation forms.....	5
2	Chapter 2 - Requirements Syntax	14
3	Chapter 3 - Characteristics of a good requirements	29
3.1	Agreed	29
3.2	Atomic or Molecular	31
3.3	Complete	43
3.4	Consistent	54
3.5	Modifiable.....	60
3.6	Traceable.....	67
3.7	Unambiguous	79
3.8	Verifiable	85
4	Chapter 4 - Writing Style	90
5	Chapter 5 - Notes Usage.....	109
6	Chapter 6 - Statement of Compliance within the requirement.....	115
7	Chapter 7 - Document sources, document owner and change history	117
8	Chapter 8 - Open points.....	121

Introduction

Document purpose

- This document presents the rules for requirements preparation of System Functional Specifications (SFS) and Entity Functional Specifications (EFS) in Nokia Mobile Networks (MN). Adherence to these rules will help ensure clear communications with other specification engineers, developers, and testers.
- The document describes golden standard how to create and update requirements written in natural language in form of:
 - single statement requirements,
 - use cases. See [Use Case definition \(see page 0\)](#)
- The document describes golden standard how to create and update following technical requirements:
 - user requirements: non-functional requirements and constraints.
 - functional requirements. See [Chapter 1 - Requirements types and requirements documentation forms \(see page 0\)](#) for user requirements and functional requirements definitions.
- In the document the term "golden standard" or abbreviation "GS" represents the document Golden Standard for Requirements Specification.
- The good specification practices defined in the golden standard are also valid for requirements written in conceptual model form as well. Example of conceptual model is UML diagram or SysML diagram.
- This golden standard principles document is created based on the references mentioned in the [Bibliography \(see page 117\)](#).
- To ensure better understanding of requirements documentation principles, the golden standard contains the examples of good and bad specification practices. (practices mentioned in 'Good examples' to be followed and practices mentioned in 'Bad examples' to be avoided)
- The golden standard shall be the foundation for internal specification guidelines prepared for all units and teams of Nokia MN business group. The internal guidelines shall be consistent with the golden standard and define the units or teams' specific requirement specification practices.

Questions, comments, proposals regarding the Golden Standard shall be reported [here](#) [Questions and answers to MN Golden Standard for Requirements Specification - MN Specification Process and WOW - Dashboard \(nokia.com\)](#)¹ **not in the Golden Standard.**

System Insights "Golden guard" AI syntax checker: <https://systeminsights.ext.net.nokia.com/golden-guard>

¹ <https://confluence.ext.net.nokia.com/display/SPECWOW/Questions+and+answers+to+MN+Golden+Standard+for+Requirements+Specification>

Table of content

- [Chapter 1 - Requirements types and requirements documentation forms \(see page 5\)](#)
- [Chapter 2 - Requirements Syntax \(see page 14\)](#)
- [Chapter 3 - Characteristics of a good requirements \(see page 29\)](#)
 - [Agreed \(see page 29\)](#)
 - [Atomic or Molecular \(see page 31\)](#)
 - [Complete \(see page 43\)](#)
 - [Consistent \(see page 54\)](#)
 - [Modifiable \(see page 60\)](#)
 - [Traceable \(see page 67\)](#)
 - [Unambiguous \(see page 79\)](#)
 - [Verifiable \(see page 85\)](#)
- [Chapter 4 - Writing Style \(see page 90\)](#)
- [Chapter 5 - Notes Usage \(see page 109\)](#)
- [Chapter 6 - Statement of Compliance within the requirement \(see page 115\)](#)
- [Chapter 7 - Document sources, document owner and change history \(see page 117\)](#)
- [Chapter 8 - Open points \(see page 121\)](#)

1 Chapter 1 - Requirements types and requirements documentation forms

Introduction

This chapter is created for information purposes only. It contains definitions of requirements, requirement types and requirements forms. The chapter is intended to ensure common understanding of the terms used in the golden standard, for all users of the document.

Definition

Definition of technical requirement in Nokia MN

For simplicity “technical requirement” is further referred in this document as “requirement”.

There are many definitions for 'requirements'. For the golden standard purpose, the below definition is used.

Requirements are a specification of what should be implemented. They are descriptions of how the system (requirement actor) should behave, or descriptions of a system property or attribute. They may be the description of a constraint on the development process of the system. [Sommerville and Sawyer]

In Nokia, MN requirements are introduced by or for a Nokia product, program/project/task(activities), and shall be associated with corresponding activity identifier (e.g. Feature-ID, Pronto-ID, CNI-ID, Jira task).

Note

The word "**requirement**" used in daily language refers to "**requirement statement**", "**requirement object**", or "**requirement text**".

However, the phrases refer to different specification artifacts*. The golden standard uses single word "**requirement**" only in the context of "**requirement statement**". Thus, the golden standard users shall follow the golden standard naming convention and use the terms correctly and consistently,

*) In DOORS tool terminology: a **requirement object** is an artifact which contains many attributes, a **requirement text** is one of the attributes, and the latter contains one or more **requirement statements**.

The term "system" referred further in this golden standard corresponds to any architecture element which belongs to Nokia MN reference architecture at any of its levels. See [BTS+RAN architecture reference model](#)².

Requirements Types

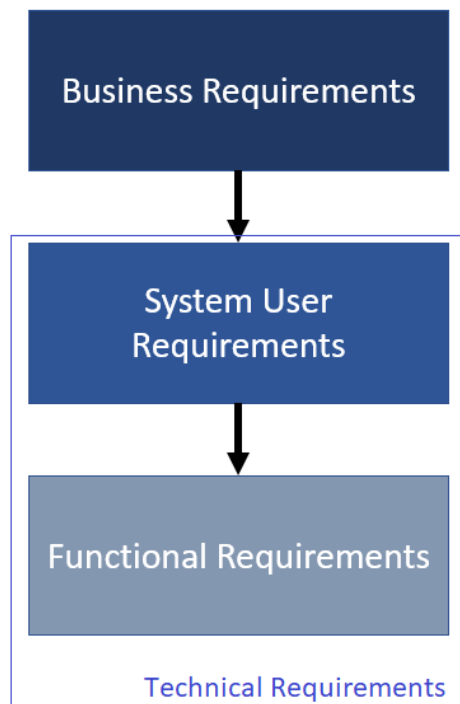
Requirement Types

There are three basic types of requirements:

- Business Requirements,
- System User Requirements,
- Functional Requirements.

Below diagram depicts the typical workflow sequence among the listed types of requirements:

² https://nokia.sharepoint.com/:p:/r/sites/5Gdoc/AS_Tech_Spec/FSYs/FSY0072020_DSS_timing_framework/BTS+RAN-architecture_simplified2021-12.pptx?d=we57afa11cea7491096be9df88d1866a6&csf=1&web=1&e=kWMQw7



The 'System User Requirements' depicted in the above diagram are further referred as 'User Requirements' in the golden standard.

Business Requirement is a high-level business objective of the organization that builds a product or of a customer who procures it. Business requirements describe why the organization is implementing the system - the business benefits the organization hopes to achieve. [Wiegers and Beatty].

User Requirement - There are three kinds of user requirements.

- user stories - it is a goal or task that specific classes of users must be able to perform with a system, or a desired product attribute. [Wiegers and Beatty].
- quality (non-functional) requirements - non-functional requirement is requirement that describes system attributes expected from the system. See: [Chapter 2 - Requirements Synta \(see page 0\)x#Requirementsyntaxforfunctionalities,qualityaspectsorconstraintstobedeliveredwithoutanyprcondition \(see page 0\)](#).
- constraints - it is a requirement that limits the solution space beyond what is necessary for meeting the given functional requirements and quality requirements. [Pohl and Rupp]. See: [Chapter 2 - Requirements Syntax \(see page 0\)](#).

There are following categories of quality requirements (see [ISO/IEC25010:2011] [Pohl and Rupp])

- Requirements that define the **performance** of the system, in particular response time behavior and resource utilization.

- Requirements that define the **security** of the system, in particular with regard to accountability, authenticity, confidentiality, and integrity
- Requirements that define the **reliability** of functionalities, in particular with regard to availability, fault tolerance, and recoverability
- Requirements that define the **usability** of a system, in particular with regard to accessibility, learnability, and ease of use
- Requirements that define the **maintainability** of a system, in particular with regard to reusability, analyzability, changeability, and testability
- Requirements that define the **portability** of a system, in particular with regard to adaptability, installability, and replaceability

The set of 6 non-functional requirements categories is not the only set that appears in Requirement Engineering literature. The golden standard does not define what non-functional requirements categories Nokia MN shall use in the requirements specification. Each Nokia MN organizational unit is allowed to choose and agree internally on the categories which best address the unit's objectives.

Examples

Examples of quality (non-functional) requirements:

- **(performance)** - System shall be able to deliver its functions according to its functional requirements consuming maximum 75% of its CPI usage.
- **(security)** - System shall have ability to encrypt configuration data stored in the BTS memory using algorithm X.
- **(reliability)** - The system availability shall be 99,999%.
- **(usability)** - Graphical User Interface shall be available for the system user.
- **(maintainability)** - The system shall ensure that 95% of its failures are possible to be resolved remotely, without the need of network engineer's physical site visit (truck-roll).
- **(portability)** - U-Plane applications shall be compatible with Loki SoC and Thor SoC.

Examples of constraints:

- The maximum total size of all BTS OAM files shall be 1GB.
- The power supply of the system shall be -48V DC.

Functional Requirement is a description of a behavior that a system will exhibit under specific conditions. [Wieggers and Beatty].

Example

Example of functional requirement:

When baseband card temperature is more than Tx value defined in [REQ:1234 in Nokia AirScale Hardware Climate Control Guideline], then BTS shall switch on secondary fan to cool the card.

Definitions added in GS v1.1 START

Functional requirements can be divided into two sub-types depending if the requirement specifies expected or exceptional actor behavior.

- **Basic requirement** - describes what is requirement actor's expected behavior.
- **Software Robustness Requirement (SW Robustness Requirement)** specifies what happens when the "basic requirement" cannot be fulfilled, for whatever reason. E.g. when a disruption happens or unwanted event.

Definitions added in GS v1.1 END

Note: To distinguish functional requirement from non-functional requirement, we can analyze if the requirement describes **WHAT CAPABILITIES** the requirement actor shall have (non-functional requirement) OR if requirement describes **WHAT** shall the requirement actor **do** (functional requirement).

Example: functional requirement vs. non-functional requirement

Imagine that you are the coach of a basketball team and would like to define functional & non-functional requirement for the new player you want to select for your team.

Functional requirements describe **what** the new player **shall do** during a basketball match:

When my team is attacking, then the new player shall try to score from the left corner of the basketball pitch.

When my team is defending, then the new player shall disturb the opponent in the midfield.

When quarterback is unavailable in my team, then the new player shall take his role and lead team in attack phase.

Non-functional requirements and constraints describe the new player **attributes, capabilities:**

The new player shall be at least 2m tall.

The new player shall have 2 years or more experience in NBA.

The new player shall be able to run at a speed of 15 km/h for 1 hour duration without break.

The average number of points scored in a match by the new player in last season shall be at least 10 points.

Note

Sometimes you may have doubts if requirement written by you is functional requirement or non-functional requirement.

It's not really a matter of concern. If the requirement is correctly written based on the golden standard, it is considered as a good requirement and can be used by software developers to build a system and testers to test it.

The example mentioned below explains:

1. how user requirements are created based on business requirements and
2. how functional requirements are created based on user requirements.

This is the most commonly followed sequence of defining requirements, but definitely not the only way.

Requirement types in an example scenario

1. Nokia customer – 'operator-X' – defines the following **business requirement**: *"Increase my end users satisfaction ratio related to specific service by 20 percent within 12 months using metric XYZ"*.

2. Operator-X realizes that the low value of 'end user satisfaction ratio' metric is due to the reason - base station recovery takes very long time once it fails. The further analysis shows that the BTS failure recovery time is long because maintenance staff of operator-X are flooded by alarms in network management system (NMS) and cannot really prioritize easily -

- which alarms have greatest impact on end users and requires immediate maintenance recovery action, and
- which alarms can wait as they have minor impact on end users.

3. Based on that, Operator-X representative together with Nokia product management defines **user requirements** written in the form of user stories.

- *As Operator, I want to have displayed only the most critical alarms in NMS alarms GUI so that the maintenance staff can start the recovery action for the critical alarms before starting recovery of minor severity alarms.*

- *As Operator, I want to have possibility to configure multiple severity and priority levels of NMS alarms, based on number of impacted end users and traffic outage measured as nominal time duration so that I can filter and display in the NMS GUI the alarms based on their severity or priority levels.*
- *As Operator, I want to have available statistics of recovery time for critical alarms in the NMS so that I can evaluate maintenance staff efficiency measured with metric ABC.*

4. Additionally, the Operator X defines another type of **user requirements** - non-functional requirements, for example:

- The NMS shall have user authentication capability for accessing GUI panels (views) for the alarm filter definition, and panel for alarm filtering results.
- The NMS version A and version B shall have alarm filtering capability.
- The NMS shall have ability to encrypt files using Advanced Encryption Standard (AES) algorithm.
- The NMS shall be available 99,9% of year time.

5. Based on the system user requirements, Nokia specification engineers prepare system **functional requirements** in the form of single statement requirements and in the form of use cases.

For example, the single statement requirements and use cases are as follows:

- *NMS operator shall have capability to select the relevant criteria to define filters needed to display the most critical alarms only. The criteria to defined filters: 1 - number of impacted end users, 2 - nominal time duration of traffic outage.*
- *NMS alarm panel (window) shall display only alarms that meet filtering criteria defined and selected by NMS operator.*
- *NMS shall collect traffic data from BTSes managed by the NMS in real time.*
- UC: Real time data collection to check how many end users are impacted by reported alarm.
- UC: Historical data collection of the network elements' failures.
- UC: Data collection of recovery actions of the network elements' failures.
- UC: Statistics generation of network elements' recovery time.
- UC: File encryption with network element's recovery statistics.

6. Additionally, based on customer user requirements, Nokia specification engineer prepares user requirements for lower level elements of the system.

- The NMS element X shall be available 99,99% of time in a year.
- The NMS element Y shall be available 99,999% of time in a year.

The golden standard covers following artifacts:

- non-functional requirements
- constraints
- functional requirements

Requirements documentation form

Requirements documentation forms

There are three forms of requirements documentation. [Pohl and Rupp]

- Requirements documentation using **natural language**
- Requirements documentation using **conceptual model** e.g.
 - use case diagram,
 - sequence diagram,
 - activity diagram,
 - state diagram
- Hybrid form of requirements documentation i.e. **combination of natural language and conceptual model**.

Note: The golden standard current version does not describe how to specify:

- interfaces in form of Messaged Definition Tables and
- requirements containing algorithms

This is planned to be added to the golden standard in future version in the near future.

Use Case definition

The term "use case" is used in the golden standard to represent requirement written in form of any sequence of steps needed to achieve any goal.

In Nokia there are various names for this kind of requirement depending on the requirement purpose or team practices. The "use case" represents e.g.: "use case", "MSC" (Message Sequence Chart), "message flow", "CPF" (Call Procedure Flow). The "Use Case" does not cover Nokia specific form "user scenario". The user scenario specification rules and good and bad examples are stored in [1. User Scenarios - guidelines](https://confluence.ext.net.nokia.com/display/SPECWOW/1.+User+Scenarios+-+guidelines)³.

The term "use case" was chosen for the golden standard as it is widely used in Requirements Engineering literature for these kind of requirements.

³ <https://confluence.ext.net.nokia.com/display/SPECWOW/1.+User+Scenarios+-+guidelines>

New part added in GS v1.1 START

Pseudo code usage

Pseudo code is not recommended as requirement form but it is not forbidden. Please note the following pseudo code drawbacks. By using other specification methods these drawbacks can be easily avoided. For requirements with complex logical formulas specification engineer can use table or activity diagram instead of pseudo code.

Pseudo code drawbacks:

- **Easy to overlook branches with unspecified behavior.** Despite of using IF ELSE AND OR, if there are complex logical formulas in IF - there is no easy visible lack of ELSE branch.
- **Inferior readability** – people read and comprehend pseudo code quite slow with a lot of focus needed to build complete, consistent, and unambiguous algorithm representation in their minds.
- **Can't easily deduct number of tests needed.** As comparison, for requirements written in tabular form one row in the table corresponds to one test. For activity diagram, single test corresponds to each branch in the diagram, where branch is a unique path from start to end.
- **There is no any standard for pseudo code.** As a result “anything goes” approach can be noticed. In existing Nokia MN specification you can find whole requirements written using IF and ELSE and indentations that are used to separate sentences and phrases of natural language. Such mix is not a pseudo code at all – it is at most “reinforced” natural language with all the drawbacks of natural language

Details of pseudo code usage in Nokia MN will be defined in the Golden Standard 2.0.

New part added in GS v1.1 END

2 Chapter 2 - Requirements Syntax

Chapter 2a

Requirement Syntax for natural language requirements

Single statement requirement

Requirements syntax for functional requirements.

The requirement shall contain following elements:

- Actor (mandatory)
- Preconditions or Trigger (or both, optional, needed when any precondition or trigger is needed to execute action described in the requirement),
- Action (mandatory)
- Observable result (optional, needed when action description is not enough to deduce the observable result of the action)

Actor = system or system element that performs action described in the requirement.

Preconditions or Trigger = "When" conjunction + description what event causes the action or what are preconditions of the action. Preconditions and trigger shall be also specified together if both are needed.

Action = What the requirement actor shall execute.

Observable result = What is observable result of the action execution.

Template

WHEN^{1, 2, 3} + [under what **CONDITION** or due to what **TRIGGER**]⁴ + , **THEN** + [requirement **ACTOR** (system name)] + **SHALL** + [process **VERB**] + [**OBJECT**] + [object **ATTRIBUTES**]⁵ with their **VALUES**⁵] + [**OBJECTIVE**]⁶

1) "When" shall be used for event-driven requirement. For state-driven requirement "While" shall be used instead. See examples below.

New point added in GS v1.1 START

2) Alternatively specification engineer can use another linking words instead of WHEN:

- **IF or**
- **IN CASE OF.**

3) Another sequence of the requirement elements are allowed if the requirement meaning is the same.

[Requirement ACTOR (system name)] + SHALL + [process VERB] + [OBJECT] + [object ATTRIBUTES with their VALUES] + [OBJECTIVE], WHEN + [under what CONDITION or due to what TRIGGER].

New point added in GS v1.1 END

4) The brackets [] are used to indicate the part of requirement is variable and it depends on specific requirement.

5) Minimum amount of all attributes and their values needed to indicate an explicit object, so the specification users can explicitly distinguish from each other different objects of the same type, and indicate the one(s), which is (are) subject to the requirement

6) A purpose, why an actor shall perform an action, i.e. a success criterion or an unambiguous short description, which allows to derive the criteria from it.

For event-driven requirement it looks like this (functional requirement, natural language, single statement requirement):

"When (After) operator initiates BTS reset," is trigger part

"then BTS OAM shall request Platform SW to collect snapshot of BTS files X and Y" is action part with actor "BOAM"

"to make the files are available in folder Z after the reset." is observable result of the action.

For state-driven requirement it looks like here (functional requirement, natural language, single statement requirement), Note the word "When" is replaced with "While".

"While BTS starts up" is precondition part

"BTS OAM shall read all detected radios calibration ports statuses" is action part with actor "BOAM"

"to report them in RMOD.calibrationPortsInUses parameter (list)" is observable result part

For requirement without trigger or preconditions it looks like here (functional requirement, natural language, single statement requirement):

The machine shall sell cold drinks.

Note.

The "shall" verb is essential in the description part. It means the requirement actor **MUST** follow the requirement.

You can find other modal verbs in e.g. 3GPP specification e.g. "should" (recommendation), "may" (permission), "can" (possibility and capability), "will" (Inevitability) and "is" (fact). 3GPP specification is different from a product specification because **standard specification** needs to allow for different compatible implementations. A product specification specifies a specific implementation, and therefore only needs the requirement term, which is **"shall"**

Hint

If you want to use another verb in a requirement, e.g. "should" mentally replace each instance of "should" with "probably won't." Would the resulting requirement be acceptable? If not, replace "should" with "shall" [Wiegers and Beatty]

Avoid verb "can" in requirements.

New point added in GS v1.1 START

If a restriction is removed sometimes specification engineer creates new requirement to clearly show that legacy restriction is not valid any longer. In the new "requirement" specification engineer uses word "can" instead of "shall". It is not allowed. The only acceptable word in single statement requirement is "shall". The following example show how to use verb "shall" instead of "can".

- **Example**

Original requirement is *OAM_REQ_CELLMGMT_123 While BTS starts up, then OAM **shall** allocate A2 DL subcells (dlSubcellType set A2) in separate L1 pool than A4 DL subcells (dlSubcellType set A4).*

When later A2 subcells can be mixed with other types of subcells due to new feature specification engineer tags the old requirement as frozen for releases before the feature delivery and creates wrong requirement with word "can". *OAM_REQ_CELLMGMT_124 While BTS starts up, then OAM **can** mix A2 DL subcells (dlSubcellType set A2) with DL subcells A4 (dlSubcellType set A4) in the same L1 Pool.*

The correct way of requirements specification is:

- **Option 1**

Specification engineer shall freeze the old requirement.*

and then specification engineer shall define new detailed atomic requirements with "shall" to cover all possible cases hidden behind the "can".

*OAM_REQ_CELLMGMT_124 When (**PRECONDITION 1**) OAM shall allocate **only A2 DL subcells** (dlSubcellType set A2) inside single L1 pool.*

*OAM_REQ_CELLMGMT_125 When (**PRECONDITION 2**) OAM shall allocate **only A4 DL subcells** (dlSubcellType set A4) inside single L1 pool.*

*OAM_REQ_CELLMGMT_126 When (**PRECONDITION 3**) OAM shall allocate **n A2 DL subcells** (dlSubcellType set A2) and **m A4 DL** (dlSubcellType set A4) subcells inside single L1 pool)*

*Freeze explanation by example: if the feature is delivered in release N, specification engineer shall specify the last release the requirement is valid for is release N-1. This way the requirement is frozen in N-1 release.

- **Option 2**

Specification engineer shall freeze the old requirement and then specification engineer shall define new molecular requirement.

*OAM_REQ_CELLMGMT_124 When (**PRECONDITION**) OAM shall allocate **A2 DL subcells** (dlSubcellType set A2) and A4 DL subcells (dlSubcellType set A4) inside single L1 pool according the rules in the table [reference to table with details of A2 and A4 allocation for various scenarios or configurations. Each row in the table has its own id].*

New point added in GS v1.1 END

Avoid verb "may" in requirements.

New point added in GS v1.1 START

Sometimes it is hard to specify when actor shall execute an action because precondition (statement after "when") is complex. In such case some specification engineers cut corners and don't specify detailed precondition and use word "may".

- Bad example

When customer configures new cell on radio connected to the BTS, BTS shall configure the cell resources on capacity card connected to the radio. The cell addition MAY require other cells configured in the BTS reconfiguration that is visible as the cells temporary drop.

Specification engineer shall define the preconditions with more details here e.g. by referencing other requirements in the precondition part.

- Good example

Two requirements covers what was covered by the bad example above. The second requirement describes when exactly actor executes the action defined after "may" in wrong requirement. *When customer configures new cell on radio connected to the BTS, then BTS shall configure the cell resources on capacity card connected to the radio.*

When according to [reference to other requirement(s)] the new cell cell resources configuration on capacity card is not possible without other cells reallocation between capacity cards then BTS shall reallocate the cells between capacity cards. The relocation will require the cells temporary drop.

New point added in GS v1.1 END

Single statement requirement

Requirement syntax for non-functional requirements and constraints

The requirement shall contain following elements:

- **Actor** (mandatory)
- **Preconditions or Trigger** (or both, optional, needed to specify the expected attribute of the actor),
- **Actor attribute** (mandatory)

For **quality (non-functional) or constraints requirements** the following requirement template shall be used:

Template

WHEN + [under what CONDITION], THEN + [requirement ACTOR (system name)] + SHALL + [VERB describing having an attribute] + [ATTRIBUTE or CHARACTERISTIC of the actor]

New point added in GS v1.1 START

Alternatively: [Requirement ACTOR (system name)] + SHALL + [VERB describing having an attribute] + [ATTRIBUTE or CHARACTERISTIC of the actor] WHEN + [under what CONDITION].

New point added in GS v1.1 END

Example 1:

- The system shall be operational in temperature range from -20 Celsius degrees to + 50 Celsius degrees

Example 2:

- The system shall have GUI in German language.

Example 3:

- When the system is located in room with temperature between 20 and 30 Celsius degrees and with humidity between 20% and 50%, then the system shall be available 99,99% of time in a year without any failure.

New point added in GS v1.1 START

Chapter 2b

Requirement Syntax for natural language requirements written as molecular requirement

Molecular requirement

Requirement syntax for molecular requirement

If group of requirements defines actor behavior for various combinations of the same input precondition parameter(s) specification engineer may create single molecular requirement instead of group separate similar requirements.

The molecular requirement may be written in the form of natural language requirement and if the number of input parameters is bigger than 2 the specification engineer shall use table to collect input parameter and actor behavior.

The molecular requirement shall contain following elements:

- Actor (mandatory)
- Preconditions or Trigger (or both, optional, needed when any precondition or trigger is needed to execute action described in the requirement),
- Action (mandatory)
- Observable result (optional, needed when action description is not enough to deduce the observable result of the action)
- Id of each actor behavior defined in the requirement

Actor = system or system element that performs action described in the requirement.

Preconditions or Trigger = "When" conjunction + description what event causes the action or what are preconditions of the action. Preconditions and trigger shall be also specified together if both are needed.

Action = What the requirement actor shall execute.

Observable result

Id

Example 1 of molecular requirement OAM_REQ_HWMGMT_5678

1. *When baseband card temperature exceeds following thresholds then BTS OAM shall take actions as below:*
 - 1.1. *The card temperature is less than or equal to Tx: switch on primary fan to cool the card*
 - 1.2. *The card temperature is more than Tx and equal or less than Ty: switch on secondary fan to cool the card*
 - 1.3 *The card temperature is more than Ty: switch off the card to protect it from damage.*

Note: Tx and Ty values are defined in REQ:1234 and REQ:5678 in Nokia AirScale Hardware Climate Control Guideline

Example 2 of molecular requirement OAM_REQ_HWMGMT_5678 *BTS OAM shall execute following actions based on baseband card temperature measurement as defined in the table*:*

Chapter 2c

Requirement Syntax for requirement written as Use Case

Requirement syntax for use case

Requirement syntax for Use Case

The requirement written as Use Case shall contain following parts:

- Title (mandatory part)
- Purpose (mandatory)
- Actors (mandatory)
- Preconditions (mandatory)
- Trigger (mandatory)
- Main Flow (mandatory)
- Alternative Flow (mandatory if exists)
- Postconditions (mandatory)
- Exception(s) (mandatory if exists)
- Notes (optional)

Title = Short information what is the scenario about.

Purpose = Brief description of the use case, use case summary.

Actors = Complete list of actors that participates in the use case. Actors not participating in the use case shall not be visible in the actors list. An actor is:

- a person that interacts with the system and
- system or system element that executes the use case steps.

Note: a person may be an actor in requirement written in form of use case. In single statement requirement the requirement actor is only system or system element.

Preconditions = Prerequisites that must be met before the system can begin executing the use case.
[Wieggers and Beatty]

Definition update in GS v1.1 START

Trigger(s) = Stimulus or list of stimuli that causes the use case execution starts if the preconditions were met."

Main Flow = List of steps executed by use case actors to evolve from the state described with precondition(s) to state described with postcondition(s).

Definition update in GS v1.1 END

Alternative Flow = Other success scenarios within the use case are called alternative flows or secondary scenarios. Alternative flows deliver the same business outcome (sometimes with variations) as the normal flow but represent less common or lower-priority variations in the specifics of the task or how it is accomplished. The normal flow can branch off into an alternative flow at some decision point in the dialog sequence; it might (or might not) rejoin the normal flow later. [Wiegiers and Beatty].

Postconditions: = A condition that describes the state of a system, after a use case is successfully completed [Wiegiers and Beatty]

Exception(s) = Rainy day scenario handling, when rainy day scenario is not specified as separate use case. This typically covers the behavior of expected actors, when any step of the use case is not executed successfully. If rainy day scenario is specified in separate use case, reference to the separate use case has to be mentioned in the Exception(s) part.

Notes (optional) = Additional notes useful to clarify any behavior described in the scenario more clearly.

Note

The Use Case parts names can differ in various Nokia MN Development Units requirements. E.g. instead of "Main Flow" the term "Steps" or "Basic flow" can be used. It is OK as long as the part with another name contains what is defined in the golden standard for corresponding part.

Example

(Title) UC: Start energy saving for LTE 4TX TDD cell

Purpose:

To switch two TXes of LTE 4TX TDD cell LNCCEL-x in order to save energy in BTS without cell deactivation.

Actors:

- NetAct

- OAM
- RROM

Preconditions

- LNCEL-x.cellTechnology is set to TDD
- The cell has commissioned 4TXes
- actOnlineTXSwitch is set to true.
- actLBRTXPowerSaving is set to true
- LNCEL-x is on LNBTS.PSGRP.lbpsCellList.
- LNCEL-x.energySavingState is switchingOffRTX.

Triggers:

RROM sends RROM_CellEnergySavingInd with parameters:

- localCellResId set y, where y is LNCEL-x.lcrId
- cellEnergySavingAction set 4 (ECellEnergySavingAction_ReduceTx))

to OAM to switch off some TXes of the LNCEL-x

Main Flow:

1.OAM deactivates 2 TX carrier of the LNCEL-x cell. (Ref: RMgmt_CarrierDeactivation)

2. OAM sends message RROM_CellEnergySavingStateChangeReq with parameters:

- localCellResId set y, where y is LNCEL-x.lcrId
- cellEnergySavingState set 2 (EOaMEnergySavingState_ReducedEnergySaving)

to RROM to change the state of the cell to Reduced Energy Saving.

3. RROM sends response RROM_CellEnergySavingStateChangeResp with parameters:

- localCellResId equal y, where y is LNCEL-x.lcrId
- status set 0 (EStatusLte_Ok))

to OAM to confirm the state of the cell is changed to “Reduced Energy Saving”.

4. OAM sends configuration change notification to NetAct with LNCEL-x.energySavingState set 2 (reducedEnergySaving)

Postconditions:

2TXes of 4TX of the LNCEL-x are switched off.

Alternative Flows

- LNCEL-x.administrativeState is set Locked
1. OAM waits for new plan from NetAct with LNCEL-x.administrativeState is set to another value than Locked.
 2. When the plan is delivered with LNCEL-x.administrativeState is set to another value than Locked OAM deactivates 2 TX carrier of the LNCEL-x cell (Ref: RMgmt_CarrierDeactivation).

Exceptions

3a RROM sends RROM_CellEnergySavingStateChangeResp with parameters:

- localCellResId equal y, where y is LNCEL-x.lcrId
 - status set to **another** value than 0
1. OAM reports major severity alarm with fault 1234 “Switching to reduced power saving cell state failed”.
 2. Use case ends, LNCEL-x.energySavingState is **not** set 2 (reducedEnergySaving)

3b RROM does not send response RROM_CellEnergySavingStateChangeResp within 30s timer.

1. OAM reports critical severity alarm with fault 1806 “Communication failure”.
2. Use case ends, LNCEL-x.energySavingState is **not** set 2 (reducedEnergySaving)

Main flow numbering scheme

The golden standard does not define single allowed scheme of main flow steps numbering. The team shall decide what numbering scheme is allowed in use cases created by them and use only the single scheme in their specification.

Examples of numbering schemes.

1. Sequence numbers

Main Flow:

1. OAM deactivates 2 TX carrier of the LNCEL-x cell. (Ref: RMgmt_CarrierDeactivation)
2. OAM sends message RROM_CellEnergySavingStateChangeReq with parameters (...)

3. RROM sends response RROM_CellEnergySavingStateChangeResp with parameters (....)

When additional step has to be added you can use natural number - dot - natural number for the step.

New part added in GS v1.1 START

The existing steps numbers shall be kept. The rationale is, the steps may be already referenced by their number from another requirement. The existing steps numbering change would require finding of all such requirements and updating the steps numbers that may be difficult, time and effort consuming.

New step number format shall be documented in Units or teams local guidelines.

Example: if the new step is added between old steps 2 and 3 the new step number may be for example 2.1 (like in the example below) or 2.A or 2a.

The local guideline shall clearly define difference between new steps numbering and sub-steps of the main step numbering.

Example: local guideline defines that number format of new step added between step N and step N+1 is N.x where x is small letter of alphabet (e.g. 2.a) whereas the step N sub steps number format is N.y where y is natural number (e.g. 2.1).

New part added in GS v1.1 END

Main Flow:

1. OAM deactivates 2 TX carrier of the LNCEL-x cell. (Ref: RMgmt_CarrierDeactivation)
2. OAM sends message RROM_CellEnergySavingStateChangeReq with parameters (...)
- 2.1. [New step content here.]
3. RROM sends response RROM_CellEnergySavingStateChangeResp with parameters (....)

2, Sequence numbers with gaps

Main Flow:

10. OAM deactivates 2 TX carrier of the LNCEL-x cell. (Ref: RMgmt_CarrierDeactivation)
20. OAM sends message RROM_CellEnergySavingStateChangeReq with parameters (...)
30. RROM sends response RROM_CellEnergySavingStateChangeResp with parameters (....)

When additional step has to be added you can use natural number between already used numbers

New part added in GS v1.1 START

The existing steps numbers shall be kept. The rationale is, the steps may be already referenced by their number from another requirement. The existing steps numbering change would require finding of all such requirements and updating the steps numbers that may be difficult, time and effort consuming.

New part added in GS v1.1 END**Main Flow:**

- 10. OAM deactivates 2 TX carrier of the LNCEL-x cell. (Ref: RMgmt_CarrierDeactivation)
- 20. OAM sends message RROM_CellEnergySavingStateChangeReq with parameters (...)
- 25. [\[New step content here.\]](#)
- 30. RROM sends response RROM_CellEnergySavingStateChangeResp with parameters (....)

3, Natural language steps labeling**Main Flow:**

- [2TX carriers deactivation]** OAM deactivates 2 TX carrier of the LNCEL-x cell. (Ref: RMgmt_CarrierDeactivation)
- [Cell energy saving request]** OAM sends message RROM_CellEnergySavingStateChangeReq with parameters (...)
- [Cell energy saving response]** RROM sends response RROM_CellEnergySavingStateChangeResp with parameters (....)

When additional step has to be added you can use new natural language label for the step.

Main Flow:

- [2TX carriers deactivation]** OAM deactivates 2 TX carrier of the LNCEL-x cell. (Ref: RMgmt_CarrierDeactivation)
- [Cell energy saving request]** OAM sends message RROM_CellEnergySavingStateChangeReq with parameters (...)
- [New natural language label here]** [\[New step content here.\]](#)
- [Cell energy saving response]** RROM sends response RROM_CellEnergySavingStateChangeResp with parameters (....)



3 Chapter 3 - Characteristics of a good requirements

Good requirement characteristics

Good requirements shall follow the below mentioned characteristics. Each characteristic is explained by at least one good and bad example of a requirement.

- [Agreed](#) (see page 29)
- [Atomic or Molecular](#) (see page 31)
- [Complete](#) (see page 43)
- [Consistent](#) (see page 54)
- [Modifiable](#) (see page 60)
- [Traceable](#) (see page 67)
- [Unambiguous](#) (see page 79)
- [Verifiable](#) (see page 85)

3.1 Agreed

Definition of the characteristic

- The requirement is AGREED when it gets approval of all stakeholders involved in the requirement specification. The approval shall be delivered in official review of the requirement specification.

Hint

Do not rely on telepathy and clairvoyance as substitutes for solid requirements specification practices. They don't work, even though they seem to be the technical foundation for some software projects. [Wiegers and Beatty]

If you are not sure if your requirement proposal reflects customer or other stakeholders needs, discuss it with the customer.

Don't assume that you know better than customer or other stakeholders of what shall be reflected in the requirement. If you are not sure how specified system shall behave for a specific case, ask the customer or stakeholder.

- Who are the stakeholders? Mentioned below are the stakeholders from E2E feature teams:
 - Higher-level specification representative who can confirm that the lower-level specification requirement follows corresponding higher-level specification requirement.

E.g. For SFS requirement, the stakeholder is Product Manager. For EFS requirement, the stakeholder is SFS specification engineer.

- Specification engineer who prepares lower-level requirement based on the higher-level requirement (e.g. for SFS requirement user is EFS specification engineer). The specification engineer can confirm the requirement is unambiguous and useful from his or her perspective.
- Developer who will implement the requirement and can confirm the requirement is feasible.
- Tester who will test the requirement can confirm the requirement is verifiable.
- Optionally it may be CuDo (Customer Documentation) representative (in case, if the requirement is used by CuDo to prepare customer documentation).
- Optionally, depending on project: other engineers, e.g. specification engineers, who need to understand specified system behavior and prepare other requirements for another project or Network Engineering engineers if the requirements have impact on their activities.

Example of lower level specification engineer (stakeholder) expectation

Let's assume requirements created by team X are valid for processor "Thor" and are tagged with "Thor".

Thor is internal component of boards ABIP and ABIQ.

If the requirement user - team Y - needs information on which requirements are valid* for ABIP and which requirements are valid for ABIQ, the team X has to add the information to their requirements.

Without this relevant information, team Y may prepare incorrect requirements.

* Valid means introduced officially by any Nokia project (feature, CNI, pronto).

- **Before the requirement specification review, specification author shall have active discussion with requirement stakeholders. The discussion is needed to exchange ideas on the requirement meaning and agree the requirement content to have the same understanding of the requirement.**
- The "Agreed" characteristic covers also requirement feasibility. It must be possible to implement each requirement within the known capabilities and limitations of the system and its

operating environment, as well as within project constraints of time, budget, and staff. [Wiegers & Beatty]

Note

In some Requirements Engineering books, it's observed that "feasible" is a separate characteristic of good requirements. In this golden standard, it is one of the conditions of another characteristic "correct".

- Even if requirement is agreed, it may require additional agreement(s) in the later phase of the project - if any stakeholder discovers the requirement to be inconsistent with another requirement or it is not doable (not feasible).

Example

Certain functionality is defined in a SFS requirement and this SFS requirement is "Agreed" by stakeholders, invited to the SFS requirement review.

During the lower-level requirement specification (e.g. EFS) phase, one of the stakeholders who didn't participate in SFS requirement definition - informs on specific hardware limitation (constraint) - which makes the SFS requirement to be not feasible.

In such case, the SFS requirement has to be re-worked and "Agreed" once again.

- The requirement that is "Agreed", is considered to be functionally correct. However, even in agreed requirement, there is a possibility of an error. Usage of the golden standard rules - helps to create correct requirement but does not guarantee the content of the requirement to be free of errors.

3.2 Atomic or Molecular

Definition of the characteristic

An ATOMIC requirement addresses one and only one aspect of system behavior.

Requirement Actor can either be compliant or non-compliant to an ATOMIC requirement. There is no possibility of partial compliance for ATOMIC requirements.

A requirement has to be either ATOMIC or MOLECULAR. The MOLECULAR requirement contains two or more ATOMIC requirements.

The MOLECULAR requirement can contain:

- option 1: reference to external document (e.g. to 3GPP document or an interface definition) & an explanation of requirements that are applicable, and not applicable (exclusions) for Nokia MN.
- option 2: copy of table stored in external document & explanation of the rows or cells of the table, that are applicable to Nokia MN
- option 3: table with requirements or set of atomic requirements - written in textual form created by the requirement author.

An ATOMIC requirement written as a Use Case, describes a sequence of steps needed to reach a single & specific goal. Each step describes only one specific action.

Conjunctions after "when"

New part added in GS v1.1 START**AND conjunction after "when" part**

Conjunction AND is allowed in precondition part (just after "When"), the requirement with the AND in precondition part still follows Atomic characteristic of good requirement.

Good example

When temperature is below 0 Celsius degrees AND the wind speed is more than 100km/h, then system shall display "Warning" message in the screen.

Note: For completeness next requirements are needed for other combinations of temperature and wind speed values. They are not here not to distract the example reader from main subject of the example (AND usage).

OR conjunction after "when" part

OR conjunction after when part is allowed in molecular requirement if the preconditions are independent and the same action is expected for all the preconditions.

Otherwise recommended form of the requirement is tabular form. See Chapter 4 - Writing Style (GS v1.1) for examples of tabular requirements with ANDs and ORs in preconditions.

Good example

The system shall display "Warning" message in the screen when 1. temperature is below 0 Celsius degrees OR 2. wind speed is more than 100km/h.

Alternatively you can create two separate atomic requirements here.

Good examples

"The system shall display "Warning" message in the screen when temperature is below 0 Celsius degrees"

"The system shall display "Warning" message in the screen when wind speed is more than 100km/h"

Complex AND and OR conjunction after "when" part

In case of complex precondition with AND and OR conjunction it is necessary to use parenthesis to define what binary operator takes precedence.

Example when the complex precondition formula result may be only 0 or 1.

Good examples

When (A OR B) AND (C OR D) is equal to 1 then system shall execute action X.

When (A OR B) AND (C OR D) is equal to 0 then system shall execute action Y.

Another example of complex precondition with AND and OR

Good example

When (A is set ON OR B is set OFF) AND (C is equal to 10MHz OR D is equal to FDD) then system shall execute action X, otherwise the system shall execute action Y.

In the requirement above there are documented 2 actions for various combinations of input parameters: action X and action Y,

If number of actions executed by requirement actor is bigger than 2 for various combinations of input parameters it is recommended to use table to collect the requirements. Single statements requirements with parenthesis are not recommended for such case.

Good example

The system shall behave as it is stated in column "System behavior" when input parameters A, B, C and D have following values.

id	A	B	C	D	System behavior
1	0	OFF	1	Yes	System shall start action E.
2	0	ON	2	No	System shall start action F.
3	0	OFF	2	No	System shall start action F.
4	1	ON	2	No	System shall start action F.

Conjunctions after "shall"

New part added in GS v1.1 START

Check if requirement part after "shall" verb contains any of the words: and, or, additionally, also, unless, except, but. If yes, and the conjunction is used to create complex sentence, probably the requirement is not atomic. For example: "When BTS shall unless.....then BTS shall.....".

OR conjunction after "shall" part

Instead of using OR conjunction after the "shall" it is better to create two separate requirements.

Bad example

When temperature is below 0 Celsius degrees then system shall display "Warning" message in the screen" if wind speed is less than 100km/h OR display "Critical alarm" if wind speed is equal or higher than 100km/h.

Good example

When temperature is below 0 Celsius degrees and wind speed is less than 100km/h , then system shall display "Warning" message in the screen"

When temperature is below 0 Celsius degrees and wind speed is equal or higher than 100km/h, then system shall display "Critical alarm" message in the screen.

AND conjunction after "shall" part

AND conjunction after "shall" part is allowed to specify all actions the requirement actor shall do when precondition happens.

Good example

"When temperature is below 0 Celsius degrees and wind speed is less than 100km/h , then system shall

- 1. display "Warning" message in the screen AND*
- 2. activate the alarm siren.*

AND conjunction after "shall" part is not allowed to specify additional example with next "when" statement.

Bad example

When temperature is below 0 Celsius degrees then system shall display "Warning" message in the screen" if wind speed is less than 100km/h AND when wind speed is equal or higher than 100km/h the system shall display "Critical alarm" in the screen.

New part added in GS v1.1 END

Atomic characteristic in Use Case

If a requirement written in the form of Use Case keeps growing in size, but new steps are still being added to the requirement main flow due to new projects/features, its recommended that the Use Case be split into smaller use cases. The reason being: lengthy use case is hard to read, hard to understand and hard to maintain - even if it still describes the sequence of actions needed to reach a single goal.

Good and bad examples for Atomic - single statement requirement

- Good example (each requirement refers to only one single temperature value exceeded)

*Requirement 1: When baseband card temperature is less than or equal to **Tx** value defined in [REQ:1234 in Nokia AirScale Hardware Climate Control Guideline], BTS OAM shall switch on primary fan to cool the card.*

*Requirement 2: When baseband card temperature is more than Tx value and equal or lower than **Ty** value defined in [REQ:1234 and REQ:5678 in Nokia AirScale Hardware Climate Control Guideline], BTS OAM shall switch on secondary fan to cool the card.*

*Requirement 3: When baseband card temperature is more than **Ty** value defined in [REQ: 5678 in Nokia AirScale Hardware Climate Control Guideline], BTS OAM shall switch off the card to protect it from damage.*

The same requirement can be written in a form of **Molecular Requirement** (single requirement containing more atomic requirement, each atomic requirement has its own id).

OAM_REQ_HWMGMT_5678

- 1. When baseband card temperature exceeds following thresholds BTS OAM shall take actions as below:*
 - 1.1. The card temperature is less than or equal to Tx: switch on primary fan to cool the card*
 - 1.2. The card temperature is more than Tx and equal or less than Ty: switch on secondary fan to cool the card*
 - 1.3 The card temperature is more than Ty: switch off the card to protect it from damage.*

Note: Tx and Ty values are defined in REQ:1234 and REQ:5678 in Nokia AirScale Hardware Climate Control Guideline

The same requirement can be also written in a form of **Molecular Requirement** with the table.

OAM_REQ_HWMGMT_5678

BTS OAM shall execute following actions based on baseband card temperature measurements:

<i>Requirement id</i>	<i>baseband card temperature</i>	<i>BTS OAM behavior</i>
<i>OAM_REQ_HWMGMT_5678 .1</i>	<i>lower than Tx</i>	<i>switch-on primary fan</i>
<i>OAM_REQ_HWMGMT_5678 .2</i>	<i>equal or higher than Tx AND lower than Ty</i>	<i>switch-on additional secondary fan</i>
<i>OAM_REQ_HWMGMT_5678 .3</i>	<i>equal or higher than Ty</i>	<i>switch-off baseband card</i>

The Tx and Ty values are defined in [REQ: 1234 and REQ:5678 in Nokia AirScale Hardware Climate Control Guideline]

- Good example (requirement refers to two actions triggered by user, where the actions have impact on each other)

Requirement 4: When machine user presses "start" and "stop" buttons at the same time, alarm "Not supported function" shall be displayed on machine screen.

- Good example (requirement refers to dynamic spectrum sharing feature only, not to any additional feature)

Requirement 5: When dynamic spectrum sharing between LTE and GSM cells is activated (LCELL.actDynamicSharedSpectrum is set "true" and LCELL.dynamicSharedSpectrumGsmCellIDN points to LCELLC representing the GSM cell) the BTS shall ensure UE attached to the LTE cell uses both LTE cell and GSM cell's downlink spectrum.

- Good example. To avoid creation of multiple similar requirements, the requirement author can create group of atomic requirements. Each of the atomic requirement, shall have its own id (label).

The three atomic requirements:

Requirement 6: BTS shall be able to transmit and receive 5G FDD cells data using OBSAI radio.

Requirement 7: BTS shall be able to transmit and receive 5G FDD cells data using CPRI radio.

Requirement 8: BTS shall be able to transmit and receive 5G FDD cells data using eCPRI radio.

can be documented as single group of atomic requirements ("**Molecular Requirement**")

Requirement 9:

1. BTS shall be able to transmit and receive 5G FDD cells data using following types of radios:

1.1 OBSAI

1.2 CPRI

1.3 eCPRI.

Good examples for high level documents with set of requirements.

The correctly written requirement that [refers to another document compliance](#) (e.g. 3GPP document or an external or internal interface definition) shall:

1. state if statements with “may” verb (instead of “shall”) in the external document are valid for Nokia MN.
2. And shall also define exceptions if any requirement in the external document shall not be followed by Nokia MN requirements.

Requirement 10 - High level requirement refers to another document with set of requirements and defines clearly which of those requirements are not valid for Nokia MN.

Req_4567

The Nokia BTS shall be able to set up calls for all valid (=not marked as N/A) transmission bandwidth configurations in 3GPP TS 38.104 V15.9.0 Table 5.3.2-1, except the combination of:

- *15MHz with SCS=60kHz*
- *50MHz with SCS=15kHz*

Comment: The excluded configurations are not tagged with requirement id (e.g. 1.1 or 1.2) as the specification engineer shall not specify negative requirements i.e. requirements including behaviors not required from the requirement author (“BTS shall NOT...”). See also [Chapter 4 - Writing Style](#) (see page 90)

Requirement 11 - High level requirement refers to another document with set of requirements and defines clearly which of those requirements in external document are not valid for Nokia MN and what other requirements replace the excluded requirements.

Req_4568

1. *The BTS shall behave as it is specified in all requirements with verb “shall” defined in 3GPP TS 12.134 V.56.7.8 excluding:*
 - 1.1. *Requirement stored in Chapter 1, point 4 regarding value of parameter X. Instead of the requirement BTS shall behave as it is specified in requirement stored in Req_7777.*
 - 1.2. *Requirement stored in Chapter 2.1, point 5.6 regarding value of timeout Y. Instead of the requirement BTS shall behave as it is specified in requirement stored in Req_7778.*

Comment: The ids are added to atomic requirements (1.1 and 1.2) in the high level requirement. The points contain exclusions BUT because the exclusions contain references to other valid requirements they are tagged with requirements ids (Req_7777 and Req_7778).

Requirement 12: High level requirement refers to another document with set of statements with “may” verb. The requirement defines clearly which of those statements with “may” shall be requirements for Nokia MN.

Req_4569

1. *The BTS shall behave as it is stated in following statements with verb "may" in 3GPP TS 12.134 V.56.7.8 i.e. the statements with "may" shall be understood as requirements with "shall":*

1.1. Statement stored in Chapter 1, point 7 regarding precoding matrix A

1.2. Statement stored in Chapter 3, point 12 regarding modulation type B.

The remaining statements with "may" in the 3GPP document are not valid for Nokia.

Comment: The ids are added to atomic requirements (1.1 and 1.2) in the high level requirement. The 3GPP documents frequently contain statements with verb "may" instead of "shall". This way 3GPP gives freedom to the document user if his/her system shall follow the statement or not.

If you refer the document in Nokia MN requirement, you shall clearly define which of the statements are requirements for MN Nokia ("may" is understood as "shall"). For clear understanding and traceability purpose tag the atomic requirements with ids (1.1 and 1.2 in the example) and use the documents internal tags (chapter number, point number, table row id or number etc.) and short clarification what is the requirement above ("precoding matrix A" "modulation type B" in the example above).

- Good example for atomic requirements **documentation**.

Requirement author can decide if single requirement management system (RMS) object, container or artifact (e.g. DOORS object) shall contain single atomic requirement or more atomic requirements. However, each of the atomic requirements shall be distinguished by its own id (label). If atomic requirement is not distinguished by id (label) there is not possibility to refer the requirement from another requirement or document. See also [Traceable](#) (see page 67).

Screenshot below shows single atomic requirements in separate DOORS objects. This is correct way of requirements documentation but not the only one that is allowed.

LTE_OP R_REQ_ 16034	SBTS shall use dedicated peer RAT (LTE or WCDMA) spectrum for peer RAT cell with dedicated bandwidth calculated based on Shared Frequency Array List (for LTE) and WBTS_UP_20455 (for WCDMA) when (i) Dynamic Spectrum Sharing (DSS) is enabled in SBTS based on requirement LTE_OPR_REQ_16028 AND (ii) the operational State of paired GSM sector is Disabled.
LTE_OP R_REQ_ 16035	SBTS shall use dedicated peer RAT (LTE or WCDMA) spectrum for peer RAT cell with dedicated bandwidth calculated based on RNL_LTE_SF5_RRM.12679 (for LTE) and WBTS_UP_20455 (for WCDMA) when (i) Dynamic Spectrum Sharing (DSS) GSM - peer RAT is configured in BTS i.e. actDynamicSharedSpectrum=true for peer RAT cell AND (ii) The peer RAT cell is not paired with any GSM sector in SBTS commissioning.
LTE_OP R_REQ_ 16036	SBTS shall use dedicated peer RAT (LTE or WCDMA) spectrum for peer RAT cell with dedicated bandwidth calculated based on Shared Frequency Array List when (i) Dynamic Spectrum Sharing (DSS) is enabled in SBTS based on requirement LTE_OPR_REQ_16028 AND (ii) Shared Frequency Array sent from BSC to SBTS in (BTS_CONF_DATA) and BITMAP sent from BSC to SBTS in DSS GSM USAGE IND indicates the peer RAT cell is NOT allowed to use shared spectrum.
LTE_OP R_REQ_ 16037	SBTS shall use dedicated peer RAT (LTE or WCDMA) spectrum for peer RAT cell with dedicated bandwidth calculated based on Shared Frequency Array when (i) Dynamic Spectrum Sharing (DSS) is enabled in SBTS based on requirement LTE_OPR_REQ_16028 AND (ii) Fault 2422: EFaultId_DssInterRatCommunicationFailureAI is detected for the peer

Here single DOORS object contains more than one atomic requirement (or groups of requirements). Each of them has its own id. This way of requirements documentation is also allowed.

Example of reference to some of the atomic requirements stored in the object:
OAM_REQ_CELL_455.1.2 or OAM_REQ_CELL_455.2.

OAM_REQ_CELL_455	1. BTS shall be able to transmit and receive 5G FDD cells data using following types of radios in terms of fronthaul protocol: 1.1 OBSAI 1.2 CPRI 1.3 eCPRI	Group of atomic requirements. Each of them has its own id (1.1, 1.2, 1.3)
	2. BTS shall be able to transmit and receive 5G TDD cells data using eCPRI radios.	Single atomic requirement with its own id (2).

If requirement defines any output value based on combination of input parameters values, recommended form of the requirement is table. Each table represents single atomic requirement and shall have its own id.

Example of reference to atomic requirement in the red frame: OAM_REQ_CELL_456.1.1.

OAM_REQ_CELL_456	1. When two alarms are active in BTS at the same time BTS shall filter them out based on following rules:					
	ID	Alarm#1 severity	Alarm#1 alarming object	Alarm#2 severity	Alarm#2 alarming object	Alarm displayed in NMS
	1.1	minor	X	minor	X	Alarm that was raised as first
	1.2	major	X	minor	X	Alarm#1
	1.3	minor	X	major	X	Alarm#2
	1.4	major	X	major	X	Alarm#1 and Alarm#2
	1.5	any	X	any	Y	Alarm#1 and Alarm#2

As the id of the requirement can be also used one or more of the input parameters. In such case reference to single atomic requirement is more complicated and specification engineer has to be very careful when specifying such reference.

Example of reference to atomic requirement in 1s row (red frame)
5G_UP_6978.NR_L1_Loki_FDD_FR1_NB_CPRI_IQF for DL

5G_UP_6978	L1 subpool composition for Loki/vDU		Direction\$	T1 L1 subpool cell slot provided\$	T2 L1 subpool cell slot provided\$	First Feature
	NR_L1_Loki_FDD_FR1_NB_CPRI_IQF		DL	48	N/A	CB007105-B
			UL	48	N/A	CB007105-B
	NR_L1_Loki_FDD_FR1_WB_CPRI_IQF		DL	64	32	CB007105-B
			UL	64	32	CB007105-B
	NR_L1_Loki_FDD_FR1_NB_CPRI_L3_CALL		DL	[Before CB009223-C]96 [CB009223-C]48 [End CB009223-C]	N/A	CB005752-A
			UL	[Before CB009223-C]96 [CB009223-C]48 [End CB009223-C]	N/A	CB005752-A
	NR_L1_Loki_FDD_FR1_NB_CPRI_5+4_CELLS		DL	40	32	CB006913-A
			UL	40	32	CB006913-A
	NR_L1_Loki_FDD_FR1_WB_NB_CPRI		DL	72	24	CB006913-C
			UL	72	24	CB006913-C

- Bad example (requirement refers to two temperature values exceeding, 55 degrees Celsius and 70 degrees Celsius)

Requirement 9: When baseband card temperature is more than T_x defined in [REQ:1234 in Nokia AirScale Hardware Climat Control Guideline], BTS OAM shall switch on secondary fan to cool the card and when the card temperature is more than T_y defined in [REQ:5678 in Nokia AirScale Hardware Climat Control Guideline], BTS OAM shall switch off the card to protect it from damage.

- Bad example (requirement refers to two services: dynamic spectrum sharing and carrier aggregation. For the example purpose assumption is the both features don't have impact on each other.)

*Requirement 10: When dynamic spectrum sharing between LTE and GSM cells is activated (LCELL.actDynamicSharedSpectrum is set "true" and LCELL.dynamicSharedSpectrumGsmCellIDN points to LCELLC representing the GSM cell) **and** at the same time carrier aggregation is activated for the LTE cell (LNBTS.actDLCAggr is set "true") the BTS shall ensure UE attached to the LTE cell uses both LTE cell and GSM cell downlink spectrum and aggregated another LTE cell downlink spectrum.*

- Bad example (high level requirement which refers to another document containing requirements. The high level requirement is bad because it does not define clearly which of those requirements from referenced document are considered valid.)

BTS shall be compliant with 3GPP TS 21.111.

- Bad example of atomic requirements **documentation** - group of atomic requirements found in DOORS. It is bad example as all the requirements are stored in single DOORS object so they have the same label (requirement id) only. None of the atomic requirements has its own id. It is impossible to separate the requirements and refer to only one individual requirement on its own from another requirement or from testing documentation. See also [Traceable](#) (see page 67).

```
[Before CB008510-A, CB006915-F]
DL subcells inside same L1 instance shall have same dlSubcellType.
UL subcells inside same L1 instance shall have same ulSubcellType.
[End CB008510-A, CB006915-F]

[Before CB008510-A]
Only on FDD dlSubcellType and ulSubcellType A2 and A4 can be mixed inside one L1 instance, otherwise all DL and UL subcells shall have same dlSubcellType and ulSubcellType.
[End CB008510-A]

[Before CB006915-F]
Only on TDD FR1 CPRI dlSubcellType and ulSubcellType C2 and C4 can be mixed inside one L1 instance, otherwise all DL and UL subcells shall have same dlSubcellType and ulSubcellType.
[End CB006915-F]

[CB008510-A]
• On FDD
  • dlSubcellType A1, A2 and A4 can be mixed inside one L1 instance
  • ulSubcellType A2 and A4 can be mixed inside one L1 instance
[End CB008510-A]

[CB006915-F]
• On TDD FR1 CPRI or TDD FR1 eCPRI:
  • dlSubcellType C2 and C4 can be mixed inside one L1 instance
  • ulSubcellType C2 and C4 can be mixed inside one L1 instance

Note: On TDD FR1 eCPRI:
  • dlSubcellType mixing or/and ulSubcellType mixing is formally supported when either CB008940 feature activation flag (i.e. MRBTS/NRBTs/actBbPoolTddFr1) is enabled or CB007113 feature activation flag (i.e. MRBTS/NRBTs/actSlowPrbPoolingTddFr1UI) is enabled, or both flags are enabled. Otherwise (i.e. when both flags are disabled):
    • all DL subcells shall have same dlSubcellType inside same L1 instance
    • all UL subcells shall have same ulSubcellType inside same L1 instance
```

- Bad example of atomic requirements **documentation**. Requirement contains table but rows in the table don't have unique ids, it is impossible to refer atomic requirement stored in single row.

OAM_REQ_CELL_457	When two alarms are active in BTS at the same time BTS shall filter them out based on following rules:				
	Alarm#1 severity	Alarm#1 alarming object	Alarm#2 severity	Alarm#2 alarming object	Alarm displayed in NMS
	minor	X	minor	X	Alarm that was raised as first
	major	X	minor	X	Alarm#1
	minor	X	major	X	Alarm#2
	major	X	major	X	Alarm#1 and Alarm#2
	any	X	any	Y	Alarm#1 and Alarm#2

Good and bad examples for Atomic - use case

- Good example - the scenario describes the sequence of steps needed to reach one single specific goal. Each of the steps describes single action.

Use case: RF module addition to activate new cell on the radio

Purpose:

To activate new cell on new radio (RMOD) connected to BTS.

Actor:

Operator, NMS (or WebEM), BTS

Preconditions:

The BTS is up and running.

Triggers:

The Operator decides to activate new cell on additional radio not connected yet to the BTS.

Main Flow:

1. The Operator plugs in the physical radio module hardware on the site.
2. BTS detects the new radio successfully.
3. The Operator performs a configuration update for the BTS from NMS (or WebEM). The following managed objects are added in the new configuration:
 - (i) RMOD
 - (ii) new cell (e.g. LNCEL) assigned to the RMOD.
4. The new configuration is taken by the BTS into use.

Postconditions:

The inserted radio module is operational.

The new cell configured on the newly inserted radio module is on air.

Exceptions:

Exc.2a In case unknown product code detected by the BTS in the RMOD, alarm with fault 4278 EFaultId_RfModuleUnauthorizedAI is reported to NMS (or WebEM) and RMOD is not taken into use. Operational state of cell(s) configured on the RMOD is Disabled.

Exc.2b In case empty product code received from the RMOD, an alarm with fault 1923 EFaultId_FrUnitUnidentified is reported to NMS (or WebEM) and RMOD is not taken into use. Operational state of cell(s) configured on the RMOD is Disabled.

Exc.2c In case incorrect or empty RMOD serial number is provided fault alarm with 1923 EFaultId_FrUnitUnidentified is reported to NMS (or WebEM). Operational state of cell(s) configured on the RMOD is Disabled.

- Bad example:
 - The scenario describes sequence of steps needed to reach more than one goal (1= cell activation on new radio, 2= BTS snapshot collection).
 - Step number 2 describes two actions.
 - All exceptions are documented in single point.

Use Case: RF module addition scenario and BTS snapshot collection**Purpose:**

To connect new radio module (RMOD) to BTS and to add the RMOD to the BTS configuration in order to enable additional cell in the BTS.

Additionally to collect BTS snapshot (logs collection) to check if no problem has happened while the radio connection.

Actors:

Operator, NMS (or WebEM), BTS

Preconditions:

The BTS is up and running.

Triggers:

The Operator decides to connect physically new radio module (RMOD) to the BTS.

Main Flow:

1. The Operator plugs in the physical radio module hardware on the site.
2. BTS detects the new radio successfully and the Operator performs a configuration update for the BTS from NMS (or WebEM). The following managed objects are added in the new configuration:
 - (i) RMOD
 - (ii) new cell (e.g. LNCEL) assigned to the RMOD.
3. The Operator sends request to BTS to collect BTS snapshot.
4. BTS collects the snapshot and uploads the snapshot to NMS (or WebEM)

Postconditions:

The inserted radio module is operational.

The new cell configured on the newly inserted radio module is on air.

The cell bandwidth is limited

Exceptions:

Exc.2a If an unknown product code is detected by the BTS in the RMOD, alarm with fault 4278 EFaultId_RfModuleUnauthorizedAI is reported to NMS (or WebEM) and RMOD is not taken into use. Operational state of cell(s) configured on the RMOD is Disabled.

In case empty product code received from the RMOD, alarm with fault 1923 EFaultId_FrUnitUnidentified is reported to WebEM/NMS and RMOD is not taken into use. Operational state of cell(s) configured on the RMOD is Disabled.

In case incorrect or empty RMOD serial number is provided, fault alarm with 1923 EFaultId_FrUnitUnidentified is reported to WebEM/NMS. Operational state of cell(s) configured on the RMOD is Disabled.

3.3 Complete

Definition of the characteristic

Complete single requirement

- A COMPLETE requirement contains all the information necessary for the reader to understand it. In the case of functional requirements, this means providing the information the developer needs to be able to implement it correctly (if the requirement level is used by developers) and tester needs to be able to test it correctly.
- Requirement is COMPLETE when all needed labels are assigned to it, contains references to tables, figures and definition of all terms used in the requirement. I.e. all needed attributes of the requirement shall be specified (e.g. hardware the requirement is valid for, software release from which the requirement is valid, feature id that introduced the requirement, higher level requirement reference - depends on Nokia MN Development Unit internal guidelines).
- If requirement **object** attributes available in requirements management system (RMS) are not enough to specify complete requirement, the requirement(s) **stored in the object** shall be supplemented with additional preconditions. For example, if requirement is valid for a specific hardware configuration (specific radio type, specific antenna array type) and RMS attribute "HW unit" is not enough to describe the configuration, the requirement(s) precondition shall contain the information on the radio type and antenna array
- Data referenced from the requirement shall be given in complete format. E.g. NIDD parameter shall be specified with at least parent managed object (MOC) Eg. LNCEL_FDD.actDISlimCarrier.
- The requirement written as single statement requirement contains all mandatory elements specified in [Chapter 2 - Requirements Syntax](#) (see page 14)
- The requirement written as use case shall consist of mandatory parts specified in [Chapter 2 - Requirements Syntax](#) (see page 22)

- If you know you're lacking certain information, use TBD (to be determined) as a standard flag to highlight these gaps, or log them in an issue-tracking system to follow up on later. Resolve all TBDs in each portion of the requirements before the developers proceed with construction of that portion. *[Wiegers and Beatty]*
- **Software robustness aspect of requirement completeness:**
 - For requirement written in use case form, COMPLETE requirement means - all identified "SW robustness" scenarios are specified as exceptions in the "basic" scenario.
 - For requirements written as single statement requirement, COMPLETE requirement means that for "basic requirement" all identified "SW robustness requirements" are specified and can be easily found in Requirement Management Systems.
 - Clear relationship between basic requirement and its SW robustness requirements shall be established. It may be e.g.:
 - RMS link between basic requirement and SW robustness requirement **(recommended option, if doable in RMS)** OR
 - parent (basic requirement) - child (SW robustness requirement) relationship in RMS structure OR
 - basic requirement and its sw robustness requirements stored in the same RMS object or artifact. In this case each requirement in the RMS object shall have manually added requirement id.
 - Note: it is allowed single SW Robustness Requirement to be assigned to more than one basic requirement. In this case the only option to group basic requirement and its SW robustness requirement is linking the requirements.

Complete set of requirements

- All following aspects shall be taken into account while preparing requirement specification
 - functionality,
 - non-functional quality attributes,
 - design constraints,
 - interfaces,
 - requirement subject response to all realizable input data, both expected input data ("basic requirement") and unexpected input data ("SW robustness requirement").

Based on the analysis specification engineer or engineers shall prepare complete set of requirements to fully reflect customer needs.

- Missing requirements are hard to spot. The following hints may help to avoid missing requirements.
 - Consider if all possible inputs, influential factors and required reactions of the system for each desired system function are specified in the requirements. This comprises describing error and exception cases.

- Symmetrical operations are a common source of missing requirements. E.g. when you specify requirement *"When system receives data: a, b and c, then the system shall validate them based on the rule..."* consider what if only a and b are delivered to the system. Shall system validate them or not? And create corresponding requirement for the case.
- Compound logical expressions often leave certain combinations of decision values. E.g. when you specify requirement *"When a=true and b=true, then the system shall...."* consider what for other a and b combinations. Decide how system shall behave when:
 - a=true and b=false or
 - a=false and b=true or
 - a=false and b=false

and create corresponding requirement.

New part added in GS v1.1 START

Completeness of requirements with logical expressions

- For requirements with logical expressions between input parameters in requirement precondition part (after "When....") specify requirement for all possible combinations of the expression parameters. Tabular form of the requirement is recommended here if number of possible behaviors is bigger than two OR number of precondition parameters is bigger than two.

New part added in GS v1.1 END

Good and bad examples for Complete - single statement requirement

- Good example 1

OAM_REQ_123

*When operator initiates BTS reset, then **BTS OAM shall** change counter of site resets for sleeping cell recovery (M801234) to 0.*

Assuming it is BOAM EFS requirement the following requirement attributes are set in BOAM EFS module in DOORS:

- Version = 1.0
- Status = Approved
- Priority = AC/M
- SW Release = 23R2
- HW Unit = ASM-6-*
- HW SW Package = ASM-6-*: 22R2
- RAN Feature = CB001234-A

- Architectural Element = CommonOAM
- Change History = 1.0 - 21.10.2022: Aleksander Rusiecki, requirement approved in CB001234-A review.
- Object Type = LCR
- Rationale = OM_GEN_SFS_SYSM_5678

Additionally the requirement is **linked** to OAM_REQ_124 (its SW robustness requirement) in RMS.

i.e. it follows [BTS OAM EFS, TRS EFS and SEC EFS Requirements Specification Guideline](#)⁴

OAM_REQ_124

*When operator initiates BTS reset and BTS OAM fails to set counter of site resets for sleeping cell recovery (M801234) to 0 then **BTS OAM** shall report alarm with fault 4321: EFaultId_CounterM801234ResetFailedAl.*

Reasons why the requirement OAM_REQ_123 is correct from Complete characteristic perspective:

- It has preconditions, action and postconditions.
- It clearly states the subject i.e. counter id is specified
- It specifies exception ("rainy day scenario")
- The requirement tagging (attributes) are specified according to valid guideline.

- Good example 2

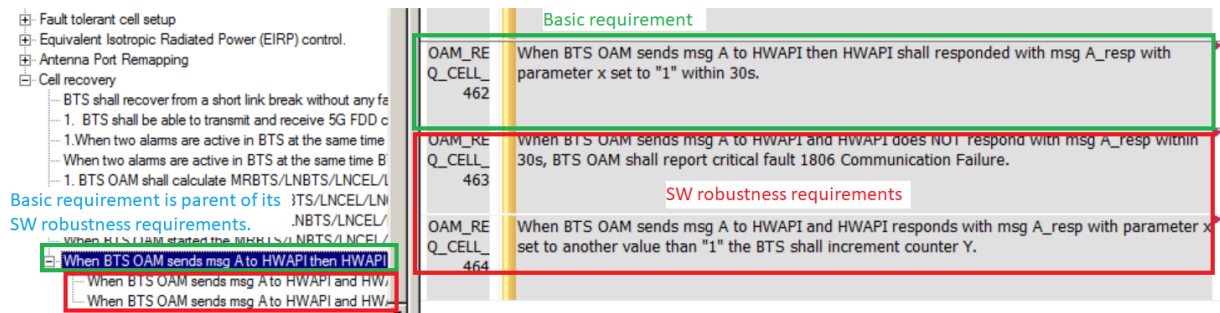
The example shows requirements completeness **from SW robustness perspective**. Requirement in green frame is basic requirement. Requirements in red frames are SW robustness requirements. There are 3 options of SW robustness requirements specification:

- option 1: Basic requirement and each SW robustness requirement stored in separate RMS objects. RMS links between basic requirement and SW robustness requirements objects created. **(recommended option, if doable in RMS)**

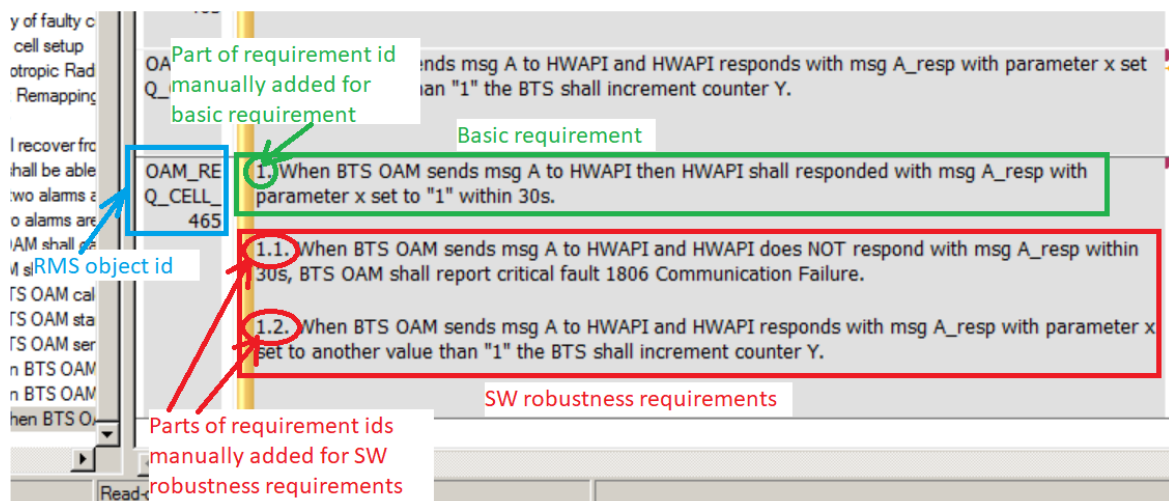
	"Disabled".	Links to sw r
	Basic requirement	
OAM_RE Q_CELL_ 462	When BTS OAM sends msg A to HwAPI then HwAPI shall responded with msg A_resp with para x set to "1" within 30s.	463: When BTS OAM sends msg A to HW... 464: When BTS OAM sends msg A to HW...
OAM_RE Q_CELL_ 463	When BTS OAM sends msg A to HwAPI and HwAPI does NOT respond with msg A_resp within 30s, BTS OAM shall report critical fault 1806 Communication Failure.	
	SW robustness requirements	
OAM_RE Q_CELL_ 464	When BTS OAM sends msg A to HwAPI and HwAPI responds with msg A_resp with parameter x se to another value than "1" the BTS shall increment counter Y.	

⁴ <https://confluence.ext.net.nokia.com/display/MANO/BTS+OAM+EFS%2C+TRS+EFS+and+SEC+EFS+Requirements+Specification+Guideline>

- option 2: The basic requirement and each SW robustness requirement stored in separate RMS objects. The basic requirement object is parent of SW robustness requirements objects in RMS structure. The user can easily find all SW robustness requirements of the basic requirement.



- option 3: The basic requirement and its sw robustness requirements stored in the same RMS object or artifact. In this case each requirement in the RMS object shall have manually added requirement id. E.g. reference to the lowest SW robustness requirement is **OAM_REQ_CELL_465:1.2**



- Note: it is allowed to assign single SW Robustness Requirement to more than one basic requirement. In this case the only option to group basic requirement and its SW robustness requirement is linking the requirements

- Good example 3

The requirement contains reference to external definition. As long as the definition is not ready, the requirement cannot be used for development and testing purpose. It is not complete.

The example contains reference to existing document with definition of "average output power", it is complete.

Req_4570

*Radio X shall be able to transmit up to 40W average output power (**Note 1**) at each air interface port.*

Note 1: The “average output power” definition is stored in Chapter 1 of document “Radio X emission algorithms.”

New part added in GS v1.1 START**Good example 4**

The requirement contains definition of "average output power" valid for the single requirement. If the definition is valid for more requirements, it shall be documented in Glossary.

Req_4570 Radio X shall be able to transmit up to 40W average output power at each air interface port. The "average output power" is calculated as sum of values of output power measurements executed every 1s within 1 minute period divided by 60

Good example 5

*When feature X is activated (NRBTS.actXfeature is set "true") and BTS detects UE type Y, then BTS shall reduce downlink transmission speed to the UE to maximum 10Mbps **otherwise the BTS shall not reduce downlink transmission speed.***

Note: The "otherwise..." part covers all remaining combinations of the requirement precondition parameters. It makes tabular form of the requirement is not needed. The remaining combinations are:

- feature X is not activated and BTS detects UE type Y OR
- feature X is not activated and BTS detects another UE type than Y OR
- feature X is activated and BTS detects another UE type than Y.

If the "otherwise..." part is missing, it is indication for the requirement user that there are other requirements that describes the requirement actor behavior for all the remaining combinations. Alternatively the requirements actor can use single requirement in tabular form to specify the requirement actor for all possible combinations of the precondition parameters.

Good example 6

BTS shall use following downlink transmission speed depending on the feature X (NRBTS.actXfeature) activation and detected UE type:

Id	NRBTS.actXfeature	UE type	DL transmission speed
1	true	Y	maximum 10Mbps
2	true	another than Y	maximum 20Mbps
3	false	Y	maximum 20Mbps
4	false	another than Y	no speed reduction

Good example 7

If there are many input parameters combinations actor behavior for all of them shall be defined in the requirement. See also **Good example 8** below. Note, there are 3 input parameters in the requirement (A,B,C) and it is created in tabular form. Creating 3 separate requirements in natural language is not recommended (for combination with id 1, with id 2 and with id 3)

When conditions A and B and C are fulfilled then actor shall behave as presented in the column "actor behavior"

id	A	B	C	actor behavior
1	1	true	on	actor does X
2	0	true	off	actor does Y

All the requirements shall have all attributes like in Good example 1 specified (it is not presented in the picture for clarity).

- **Bad example 1**

*When operator initiates BTS reset, then **BTS OAM shall** update site resets for sleeping cell recovery counter.*

Assuming it is BOAM EFS requirement the following requirement attributes are set in BOAM EFS module in DOORS:

- Version = 1.0
- Status = Approved
- SW Release = 23R2
- HW Unit = ASM-6-*
- RAN Feature = CB001234-A
- Architectural Element = CommonOAM
- Change History = 1.0 - 21.10.2022: Aleksander Rusiecki, requirement approved in CB001234-A review.
- Object Type = LCR

i.e. it doesn't follow [BTS OAM EFS, TRS EFS and SEC EFS Requirements Specification Guideline](#)⁶

Reasons why the requirement is incorrect from Complete characteristic perspective:

- It doesn't clearly state the subject i.e. counter id is not specified, what does it mean "update"?
- Not all needed* requirement attributes are specified.
- needed = required by specification guideline valid for impacted DU (development unit)

⁶ <https://confluence.ext.net.nokia.com/display/MANO/BTS+OAM+EFS%2C+TRS+EFS+and+SEC+EFS+Requirements+Specification+Guideline>

New part added in GS v1.1 START

Bad example 2

When feature X is activated (NRBTS.actXfeature is set "true") and BTS detects UE type Y, then BTS shall reduce downlink transmission speed to the UE to maximum 10Mbps. (Assumption is that other requirements that don't cover remaining combinations of the precondition input parameters don't exist). Why it is bad example? It is not stated what shall happen when:

- feature X is not activated and BTS detects UE type Y OR
- feature X is not activated and BTS detects another UE type than Y OR
- feature X is activated and BTS detects another UE type than Y.

Shall BTS not limit the speed at all? Or maybe shall limit to 20Mbps? Or maybe behave another way?

Bad example 3

When A is equal to 1 and B is equal to "true" and C is set to "on", then system shall do X". Why it is bad example? There are more than 2 parameters values in preconditions and the requirement shall be created in tabular for together with other combinations of the input parameters A, B and C.

Bad example 4

The bad example of huge number of restrictions valid for specific group of input parameters. Example for specification of huge number of Carriers Combinations allowed for configuration defined as RAT type + Number of Carriers + Number of DL and UL streams per carrier.

NR only	3CC	16DL/8UL	Any combination within oBW (CB012725-CA)
---------	-----	----------	--

Why it is bad example? For the reader it is unclear what does the (CB012725-CA) mean in the context of the restriction.

New part added in GS v1.1 END

Good and bad examples for Complete - use case

- Good example

Use case: RTWP fault reporting.

Purpose:

To report alarm with fault "RTWP level too low" based on RTWP measurements if RTWP level is too low.

Actors:

BTS, NMS, Operator

Preconditions:

1. RX monitoring is enabled in BTS by setting the BTSSCL.actRxMonitoring to "true"
2. BTS periodically monitors RTWP levels of uplink signals received by RX antennas of radios connected to the BTS. The monitoring is executed separately for each RX antenna. For details

see Use case: RTWP monitoring.

Trigger:

Any hardware problem happens on RX antenna or in any hardware transmitting RX data from antenna in BTS hardware resources.

Main Flow:

1. BTS detects that RTWP level is below threshold `BTSSCL.RTWPTreshold` on any of the RX antennas and the too low value is observed constantly for 30 minutes. The too low RTWP value shall be treated as RX critical fault.
2. BTS calculates impact of the too low RTWP RX failure to operational state of cell or cells that use the faulty RX antenna or hardware transmitting data from the RX antenna.
3. BTS raises alarm informing operator about cell(s) failure or degradation due to RX failure and sends the alarm to NMS.

Postconditions:

Operator is informed about about RX failure with alarm with fault "RTWP level too low" in NMS. The alarm shows what RX antenna and what cells are impacted by the RX failure.

Impacted cell(s) operational state is reported in NMS as Disabled if the RX failure makes the cell is not able to be operational any longer or Enabled (and availability Status Degraded) in opposite case. The rules of operational state calculation is stored in `OM_GEN_SFS_SYSM_12345`.

Exceptions:

Exc.1a RTWP measurements are unavailable due to any BTS failure. BTS raises alarm informing operator about RTWP measurement issue. The alarm shows for what RX antenna the measurement is not executed.

Assuming it is team X requirement the following requirement attributes are set in X SFS module in DOORS:

- Requirement Version = 1.0
- Priority = AC/M
- System Release = 23R2
- RAT Feature = CB001234-A
- Requirement Type = FUN
- Change History = 1.0 - 1.02.2023: Aleksander Rusiecki, requirement approved in CB001234-A review.

i.e. it follows X specification guideline

Reasons why the requirement is correct from Complete characteristic perspective:

- It contains all mandatory parts (see [Chapter 2 - Requirements Syntax \(see page 22\)](#))
- It clearly states the subject i.e. objects names and parameters names are used, fault name is used as well, it contains detailed reference to another requirement.
- It specifies clearly when alarm shall be reported.
- It specifies exception (SW robustness part of the requirement).
- The requirement tagging (attributes) are specified according to valid guideline.

- Bad example

Use Case: RTWP fault reporting.

Actors:

BTS

Preconditions:

BTS is operational.

Trigger:

Any hardware problem happens on RX antenna or in any hardware transmitting RX data from antenna in BTS hardware resources.

Main Flow:

1. BTS detects that RTWP level is below threshold on any of the RX antennas. The too low RTWP value shall be treated as RX critical fault.
2. BTS calculates impact of the too low RTWP RX failure to operational state of cell or cells that use the faulty RX antenna or hardware transmitting data from the RX antenna.
3. BTS raises alarm informing operator about cell(s) failure or degradation due to RX failure and sends the alarm to NMS.

Post-conditions:

Operator is informed about about RX failure with alarm with fault "RTWP level too low" in NMS. The alarm shows what RX antenna and what cells are impacted by the RX failure.

Assuming it is team X requirement the following requirement attributes are set in X SFS module in DOORS:

- Priority = AC/M
- System Release = 23R2
- RAT Feature = CB001234-A
- Change History = 1.0 - 1.02.2023: Aleksander Rusiecki, requirement approved in CB001234-A review.

Reasons why the requirement is incorrect from Complete characteristic perspective:

- It does NOT contain all mandatory parts (see [Chapter 2 - Requirements Syntax \(see page 22\)](#)); "Purpose" is missing.
- It does NOT clearly states the subject i.e. objects names and parameters names are not used to define "too low RTWP" or "threshold".
- The Preconditions is too general, it does NOT contain RX monitoring activation flag value.
- It doesn't specify when the fault "RTWP level too low" shall be reported.
- It does NOT specify fully what is impact of specified fault on cell(s) operational state.
- It does NOT specify exception (missing "SW robustness" part of the requirement).
- The requirement tagging (attributes) is NOT specified according to valid guideline.

3.4 Consistent

Definition of the characteristic

Functional aspect of the CONSISTENT characteristic:

The CONSISTENT requirement must not be in conflict with other requirements on the same or higher level. E.g. EFS shall be consistent with SFS.

Non-functional aspects of the CONSISTENT characteristic:

- The requirement shall be stored in correct place following internal rules defined by specific team.
- Requirement shall follow "BTS RAN architecture framework and terminology" E.g. actors presented in the requirement shall be consistent with the requirement level. https://nokia.sharepoint.com/:p:/r/sites/5Gdoc/_layouts/15/Doc.aspx?sourcedoc=%7BE57AFA11-CEA7-4910-96BE-9DF88D1866A6%7D&file=BTS%2BRAN-architecture_simplified2021-12.pptx&wdLOR=c2C2888CF-D502-4AC9-8F7C-777086289EF5&action=edit&mobileredirect=true&cid=d4e590fd-ce33-44ca-b48f-c3cc682ef75b
- The same requirement actor or the same requirement object shall have the same name in all requirements stored in the same specification repository. Synonyms usage is forbidden. E.g. don't use "path", "route", "channel" names for the same object. Repository means all modules, folders used by specific level specification engineers (e.g. by SFS engineers) of the same Nokia MN development unit (e.g. L3).

Correction introduced in GS v1.1 START

- If a decision is taken in Nokia MN to change name for any software component or software element or hardware type or architecture element e.g from X to Y, the new name shall be used in new requirements together with old name. For example in legacy requirements the name is "X", in new requirements the name shall be "Y(X)". both names shall be explained in requirements repository glossary.
- The way the change is specified shall give the req reader opportunity to observe what was old name, what is new name and when the old was replaced with new names.
- The new name shall be used in new version of the requirement. Previous versions of the requirement shall still contain old name. In the future, if the new version of the requirement is changed, only the new name shall be used in the requirement. The Golden Standard does not specify details how to show any name change in requirement content. Each MN team shall decide it on its own and document in the team guideline for requirements specification.

Correction introduced in GS v1.1 END**Good and bad examples for Consistent - single statement requirement**

- **Good example 1 (two requirements are consistent as they both mention the report shall be in tabular form)**

Requirement 1:

When Country Data System user inputs text file with countries and their capital cities separated by comma, then the Country Data System shall put capital cities names into table "Countries Data" into column "Capital City" and display the table on the screen.

Requirement 2:

When Country Data System user inputs text file with countries and their currencies separated by comma, then the Country Data System shall put currencies into table "Countries Data" into column "Currency" and display the table on the screen.

New part added in GS v1.1 START

Good example 2

The example shows how message name change is documented in single statement requirement due to feature CB008326-C. Note, this is not the only right way how the name change can be documented. Each team can decide on its own how to document the name change in the way that is easy to understand.

5G_L3_FLOW_3402	(7.A1.2) Step: CplNb_NgLinkInfoUpdateInd Direction: 5G-CP-IF to 5G-CP-NB Message Definition: [5G_L3_IF_44]	5G-CP-IF	5GC000740-E: 5G20A// : CP2 Approved
5G_L3_FLOW_3403	5G-CP-IF shall send CplNb_NgLinkInfoUpdateRequired message to 5G-CP-NB with the AMF action IE set to "modify" and the linkStatus IE set to "INACTIVE".	5G-CP-IF	5GC000740-B: 5G20A// : CP2 Approved 5GC000740-E: 5G20A// : CP2 Approved
5G_L3_FLOW_24894	5G-CP-IF shall send (Before CB008326-C)CplNb_NgLinkInfoUpdateRequired (CB008326-C) CplNb_NgLinkInfoUpdateInd (End CB008326-C) message to 5G-CP-NB with the AMF action IE set to "modify" and the linkStatus IE set to "INACTIVE".	5G-CP-NB 5G-CP-IF	CB008326-C: //2R2-CR/ : CP2 Approved

New part added in GS v1.1 END

- **Bad example**

Requirement 1:

*When system Country Data System user inputs text file with countries and their capital cities separated by comma, then the Country Data System shall put capital cities names into **table "Countries Data"** into column "Capital City" and display **the table** on the screen.*

Requirement 2:

*When Country Data System user inputs text file with countries and their currencies separated by comma, then the system Country Data System put currencies into **text file "Countries Data"** and display the **text file** content on the screen*

Why it is bad example?

Two requirements are inconsistent as one of them requires the report in tabular format while the second requires the report in text format.

Good and bad examples for Consistent - use case

The consistency of two use cases is presented based on

- **Use Case: RTWP fault reporting**, good example in the chapter [Complete](#) (see page 43) and
- **Use Case: RTWP fault clearance**, see below

- **Good example 1**

The scenario is consistent with **Use Case: RTWP fault reporting** as:

- the same parameter names and object names are used in both scenarios
- the same fault name is used in both scenarios
- the same impact of fault on cells is specified in both scenarios

Use Case: RTWP fault clearance.

Purpose: To clear alarm with fault “RTWP level too low” based on RTWP measurements if RTWP level is equal to or above threshold configured by customer.

Actors:

BTS, NMS, Operator

Preconditions:

1. RX monitoring is enabled in BTS by setting the BTSSCL.actRxMonitoring to "true".
2. BTS periodically monitors RTWP levels of uplink signals received by RX antennas of radios connected to the BTS. The monitoring is executed separately for each RX antenna.
3. Alarm with fault “RTWP level too low” is active for any of RXes in the BTS.
4. Operational state of cells impacted by the “RTWP level too low” fault is either Disabled if the RTWP too low fault makes the cell not able to be operational any longer or Enabled (and availability status Degraded) if cell can still be operational but degraded.
5. There is not active any other fault in the BTS.

Trigger:

The problem on RX antenna or in any hardware transmitting RX data from antenna in BTS hardware resources is removed.

Main Flow:

1. BTS detects that RTWP level is equal to or above threshold BTSSCL.RTWPThreshold on the RX antenna for more than 10 minutes.
2. BTS clears the “RTWP too low” fault for the RX.
3. BTS calculates impact of the canceled “RTWP to low fault” on operational state of cell or cells that use the faulty RX antenna or hardware transmitting data from the RX antenna. The rules of operational state calculation is stored in **OM_GEN_SFS_SYSM_12345**.
4. BTS sends alarm cancel with fault “RTWP too low fault” to NMS.

Postconditions:

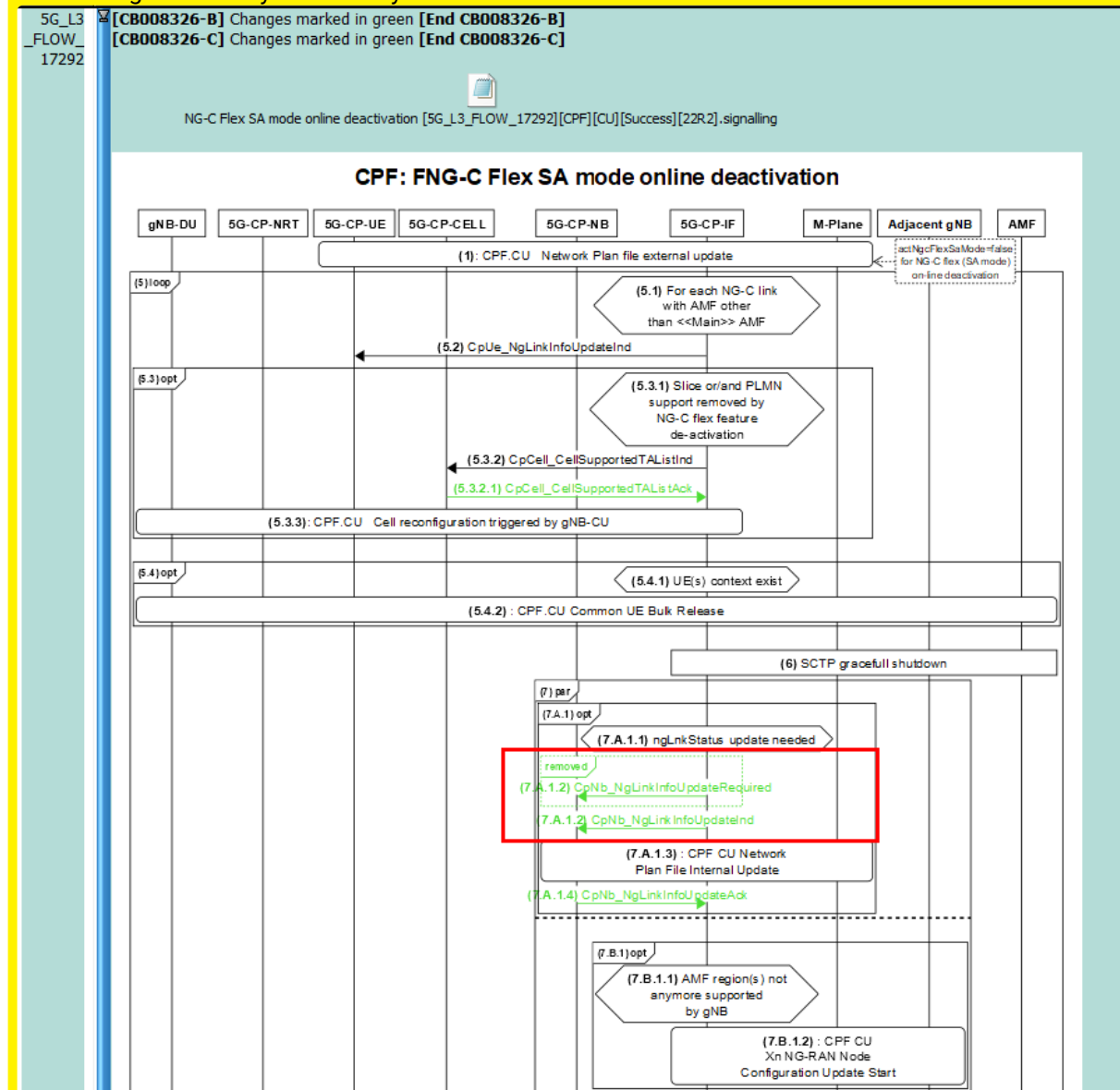
Operator is informed about cancellation of alarm with fault “RTWP level too low” in NMS.

Impacted cell(s) operational state is reported in NMS as Enabled. None of cells using the RX antenna has availability status Degraded.

New part added in GS v1.1 START

Good example 2

The example shows how message name change can be documented. Note, this is not the only right way how the name change can be document. Each team can decide on its own how to document the name change in the way that is easy to understand.



The green fonts show changes introduced due to new feature. In the red frame you can find old requirement name marked as "removed" and the new requirement name just below the old, removed name.

New part added in GS v1.1 END

• Bad example

The scenario is inconsistent with **Use Case: RTWP fault reporting** as:

- different parameters names and object names are used in both scenarios.
- Different fault names are used in the two scenarios.
- Different impact on cells of the fault is specified in both scenarios.

The scenario is also inconsistent internally because:

- different fault name is used in various parts of the scenario
- one of actors has different name in the Actors part and in Postconditions part.

Use Case: RTWP fault clearance

Purpose:

To clear alarm with fault **"RX failure"** based on RTWP measurements if RTWP level is equal to or above threshold configured by customer.

Actors:

BTS, **NetAct**, Operator

Preconditions:

1. RX monitoring is enabled in BTS by setting the **MNL.actRxMonitoring** to "true".
2. BTS periodically monitors RTWP levels of uplink signals received by RX antennas of radios connected to the BTS. The monitoring is executed separately for each RX antenna.
3. Alarm with fault **"RX failure"** is active for any of RXes in the BTS.
4. Cells impacted by the **"RTWP level too low"** fault operational state is **Enabled** and their availability status is **Degraded**.
5. There is not active any other fault in the BTS.

Trigger:

The problem on RX antenna or in any hardware transmitting RX data from antenna in BTS hardware resources is removed.

Main flow:

1. BTS detects that RTWP level is equal to or above threshold **MNL.RTWPThreshold** on the RX antenna for more than 10 minutes.
2. BTS clears the "RTWP too low" fault for the RX.
3. BTS calculates impact of the canceled "RTWP to low fault" on operational state of cell or cells that use the faulty RX antenna or hardware transmitting data from the RX antenna. The rules of operational state calculation is stored in **OM_GEN_SFS_SYSM_12345**.
4. BTS sends alarm cancel with fault "RTWP too low fault" to NMS.

Postconditions:

Operator is informed about cancellation of alarm with fault "RTWP level too low" in **NMS**.

Impacted cell(s) operational state is reported in NMS as Enabled. None of cells using the RX antenna has availability status Degraded

3.5 Modifiable

Definition of the characteristic

- A requirements specification is MODIFIABLE, if its structure and style are such that any changes to the requirements can be made easily, completely, and consistently while retaining the structure and style.
- The MODIFIABLE requirement shall have its own labels (requirement ids) and the label shall be used to reference one requirement from another. This way if you change single requirement you can find all dependent requirements and decide if they need any modification or not.
- If the requirement's postconditions depend on many preconditions (any value is selected based on a few parameters), it is recommended to use tabular form for the requirement. Each row of the table will denote a separate requirement (sub-requirement).
- If any table or diagram is part of the requirement, the source file with the table or diagram shall be available. Do not use screenshot of the table or diagram in the requirement! The specification engineer will waste time to create if from the scratch when the requirement needs to be modified. Additionally it is impossible to find any word or number in the screenshot by search function in requirements management system (RMS).
- The MODIFIABLE requirement shall not repeat another requirement or part of another requirement. Requirements duplication makes it is difficult to maintain their consistency.
- When any requirement is modified, specification engineer shall clearly describe which part of the requirement is modified and highlight it in the change description (e.g. Change History attribute in DOORS).
- After the modification, the requirement shall still fulfill all the characteristics of a good requirement

Good and bad examples for Modifiable - single statement requirement

- Good example 1

When feature slow PRB polling is not activated (NRBTS.actSlowPrbPooling is set to "False") in BTS and the CPUs usage on baseband cards in BTS is lower than 50% [Note 1], then after the slow PRB pooling activation (MNL.actSlowPrbPooling is set to "true") the BTS shall ensure CPUs usage is lower than 70% [Note 2].

If the CPU usage is 70% or more BTS shall report alarm with fault 8765: EFaultId_CpuOverload.

Note 1: The condition is fulfilled when the usage is lower than 50% for at least 99% of the time when BTS is powered on, counted each hour.

Note 2: The condition is fulfilled when the usage is lower than 70% for at least 99% of the time when BTS is powered on, counted each hour.

Why the requirement is modifiable?

- It follows requirement syntax pattern: precondition/trigger + description + postconditions (see: [Chapter 2 - Requirements Syntax \(see page 0\).](#))
- The complicated definition of the CPU usage is excluded from the requirement content - it makes it is easy to update the definition if needed in the future.
- The exception i.e. what happens if the system goes above 70%, is specified in separate sentence.

- Good example 2

When the requirement presented above requires modification, the requirement change history clearly describes what was changed in the requirement.

When feature slow PRB polling is not activated (NRBTS.actSlowPrbPooling is set to "False") in BTS and the CPUs usage on baseband cards in BTS is lower than 60% [Note 1], then after the slow PRB pooling activation (MNL.actSlowPrbPooling is set to "true") the BTS shall ensure CPUs usage is lower than 70% [Note 2].

If the CPU usage is 70% or more BTS shall report critical severity alarm with fault 8765: EFaultId_CpuOverload.

Note 1: The condition is fulfilled when the usage is lower than 60% for at least 99% of the time when BTS is powered on, counted each hour.

Note 2: The condition is fulfilled when the usage is lower than 70% for at least 99% of the time when BTS is powered on, counted each hour.

Change history:

21.6.2023: Aleksander Rusiecki: Changes made in CB001234-A

- 50% replaced with 60% in "(...) and the CPUs usage on baseband cards in BTS is lower than 60% (...)" and in Note 1.
- "critical severity" added to "(...) BTS shall report critical severity alarm with fault 8765(...)"

- Good example 3

EFS requirement OAM_REQ_CELL_1234 does NOT duplicate formula defined in SFS requirement OM_GEN_SFS_FM_5678

OM_GEN_SFS_FM_5678

When feature X is activated (actFeatureX is set "true") then while BTS startup BTS shall calculate parameter_y value based on formula [parameter_y = a+b] and send parameter_y value to NetAct

OAM_REQ_CELL_1234

*When feature X is activated (actFeatureX is set "true") and Component_B reports RMOD operational state is Enabled to BTS_OAM then BTS OAM shall calculate parameter_y based on formula defined in **OM_GEN_SFS_FM_5678** and send parameter_y value to NetAct.*

- Bad example 1

When feature slow PRB polling is not activated (NRBTS.actSlowPrbPooling is set to "false") in BTS and the CPUs usage on baseband cards in BTS is lower than 50% for at least 99% of time when BTS is powered on, counted each hour, then after the slow PRB pooling activation (MNL.actSlowPrbPooling is set to "true") if the CPU usage is bigger then 70% then BTS shall report alarm with fault 8765: EFaultId_CpuOverload as the usage shall be lower than 70% for at least 99% of the time when BTS is powered on, counted each hour.

Why is this a bad example?

- As definition of CPU usage is embedded into requirement content, it makes the requirement complicated and hard to modify - when the definition needs to be changed in the future.
- Similarly, the exception case is inside the requirement content. It makes it difficult to change the exception part in the future if e.g. besides alarm reporting, additionally any recovery action shall be executed due to CPU overload.

- Bad example 2

Requirement is difficult to modify when it contains embedded table or diagram, and the source document of the table or diagram is not available. Any change in the table or diagram requires it to be recreated from the scratch.

Recommendation: add source document to the requirement (e.g. xls file or visio file or plantUML file) or use RMS tool feature that allows to modify the requirement content (e.g. DOORS table).

- Bad example 3

EFS requirement OAM_REQ_CELL_1234 duplicates formula defined in SFS requirement OM_GEN_SFS_FM_5678

OM_GEN_SFS_FM_5678

*When feature X is activated (actFeatureX is set "true") then while BTS startup BTS shall calculate parameter_y value based on formula **[parameter_y = a+b]** and send parameter_y value to NetAct*

OAM_REQ_CELL_1234

*When feature X is activated (actFeatureX is set "true") and Component_B reports RMOD operational state is Enabled to BTS_OAM then BTS OAM shall calculate parameter_y based on formula **[parameter_y = a+b]** and send parameter_y value to NetAct.*

Good and bad examples for Modifiable - use case

- Good example

The requirements presented in the example are modifiable as:

- Table added to requirement OM_GEN_SFS_FM_1234 can be modified by requirement author (it is not just a screenshot of the table).
- There is no duplication of the requirement OM_GEN_SFS_FM_1234. The table with filtering rules is only in OM_GEN_SFS_FM_1234. OM_GEN_SFS_FM_9876 only refers to the table.
- The use case in OM_GEN_SFS_9876 follows the use case syntax stored in the [Chapter 2 - Requirements Syntax](#) (see page 22)
- The requirements are correctly labelled. Use case has its own label (OM_GEN_SFS_FM_9876) and the requirement written as single statement also (OM_GEN_SFS_FM_1234). Additionally, each sub-requirement in the table in OM_GEN_SFS_FM_1234 has its own label (OM_GEN_SFS_FM_1234.1, OM_GEN_SFS_FM_1234.2 etc.)

Use Case: Alarms filtering (

OM_GEN_SFS_FM_9876
)

Purpose:

The use case describes how BTS filters alarms in order not to flood customer with high number of alarms.

Actors

Operator, BTS, NMS

Preconditions

- BTS is on air.
- No fault is active in the BTS.
- MPlane connection between BTS and NMS is established

Trigger

- Any fault (fault#1) is detected in the BTS.

Main Flow

1. The BTS calculates alarm#1 type and alarm#1 severity to be reported to NMS due to the fault#1. The calculation is executed based on NIDD description of the fault#1.
2. The BTS reports the alarm#1 due to fault#1 to NMS.
3. Another fault is detected in the BTS – fault#2.
4. The BTS calculates alarm#2 type and alarm#2 severity to be reported to NMS due to the fault#2. The calculation is executed based on NIDD description of the fault#2.
5. BTS executes alarm#1 and alarm#2 filtering based on rules stored in **OM_GEN_SFS_FM_1234**.

Alternative Flow 1: [The alarm correlation result is both alarms shall be visible for customer]

The BTS reports the alarm#2 due to fault#2 to NMS.

End of Alternative Flow 1

Alternative Flow 2: [The alarm correlation result is only alarm#1 shall be visible for customer]

BTS suspends alarm#2 reporting to NMS, only alarm#1 is visible in NMS

End of Alternative Flow 2

Alternative Flow 3: [The alarm correlation result is only alarm#2 shall be visible for customer]

BTS sends alarm#1 cancel to NMS and reports alarm#2 to NMS. Only alarm#2 is visible in NMS.

End of Alternative Flow 3

Postconditions

Only alarms not filtered out based on rules stored in

OM_GEN_SFS_FM_1234

are visible for Operator in NMS.

Exceptions

EXC.2a. The MPlane connection to NMS is lost

BTS follows fault management rules stored in

OM_GEN_SFS_FM.5678

.

EXC.6a. The MPlane connection to NMS is lost

BTS follows fault management rules stored in

OM_GEN_SFS_FM.5678

.

And another requirement referenced from previous use case:

Alarm filtering rules (

OM_GEN_SFS_FM_1234

)

When two alarms are active in BTS at the same time BTS shall filter them out based on following rules:

Requirement ID	Alarm#1 severity	Alarm#1 alarming object	Alarm#2 severity	Alarm#2 alarming object	Alarm displayed in NMS
OM_GEN_SFS_FM_1234.1	minor	X	minor	X	Alarm that was raised as first
OM_GEN_SFS_FM_1234.2	major	X	minor	X	Alarm#1
OM_GEN_SFS_FM_1234.3	minor	X	major	X	Alarm#2
OM_GEN_SFS_FM_1234.4	major	X	major	X	Alarm#1 and Alarm#2
OM_GEN_SFS_FM_1234.5	any	X	any	Y	Alarm#1 and Alarm#2

Note: List of alarming objects is stored in

OM_GE_SFS_FM_8822

- Bad example

The requirements presented in the example are difficult to modify as:

- Table added to requirement OM_GEN_SFS_FM_1234, cannot be modified by requirement author, as it is the table screenshot. The same is the case for the table in OM_GEN_SFS_FM_9876.
- The table with alarm filtering rules is duplicated as it is part of OM_GEN_SFS_FM_1234 and OM_GEN_SFS_FM_9876. Moreover, in OM_GEN_SFS_FM_9876 there is no reference to the OM_GEN_SFS_FM_1234, and vice versa. If anybody changes the table in one of the requirements in the future, it will not be informed that the same table is also used in another requirement and the tables content will be inconsistent across both requirements.
- The sub-requirements in the table in OM_GEN_SFS_FM_1234 - don't have their own labels.

Use Case: Alarms filtering (OM_GEN_SFS_FM_9876)

Purpose:

The use case describes how BTS filters alarms in order not to flood customer with high number of alarms.

Actors

Operator, BTS, NMS

Preconditions

- BTS is on air.
- No fault is active in the BTS.
- MPlane connection between BTS and NMS is established

Trigger

- Any fault (fault#1) is detected in the BTS.

Main Flow

1. The BTS calculates alarm#1 type and alarm#1 severity to be reported to NMS due to the fault#1. The calculation is executed based on NIDD description of the fault#1.
2. The BTS reports the alarm#1 due to fault#1 to NMS.
3. Another fault is detected in the BTS – fault#2.
4. The BTS calculates alarm#2 type and alarm#2 severity to be reported to NMS due to the fault#2. The calculation is executed based on NIDD description of the fault#2.
5. BTS executes alarm#1 and alarm#2 filtering based on rules stored in the table



Alternative Flow 1 [The alarm correlation result is both alarms shall be visible for customer]

The BTS reports the alarm#2 due to fault#2 to NMS.

End of Alternative Flow 1

Alternative Flow 2 [The alarm correlation result is only alarm#1 shall be visible for customer]

BTS suspends alarm#2 reporting to NMS, only alarm#1 is visible in NMS

End of Alternative Flow 2

Alternative Flow 3 [The alarm correlation result is only alarm#2 shall be visible for customer]

BTS sends alarm#1 cancel to NMS and reports alarm#2 to NMS. Only alarm#2 is visible in NMS.

End of Alternative Flow 3

Postconditions

Only alarms not filtered out based on rules stored in **OM_GEN_SFS_FM_1234** are visible for Operator in NMS.

Exceptions

EXC.2a. The MPlane connection to NMS is lost

BTS follows fault management rules stored in **OM_GEN_SFS_FM.5678**.

EXC.6a. The MPlane connection to NMS is lost

BTS follows fault management rules stored in **OM_GEN_SFS_FM.5678**.

And another requirement referenced from previous use case:

Alarm filtering rules (OM_GEN_SFS_FM_1234)

When two alarms are active in BTS at the same time BTS shall filter them out based on following rules:



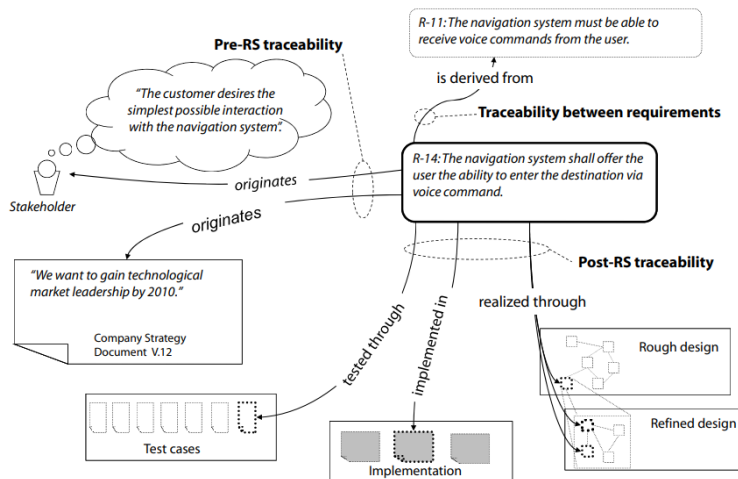
Note: List of alarming objects is stored in **OM_GE_SFS_FM_8822**.

3.6 Traceable

Definition of the characteristic

Traceability types

- There are 3 types of requirements TRACEABILITY. They are explained in the diagram below. Source: [Pohl and Rupp]



- (1) Pre-RS (pre-requirement specification) traceability ensures business and user requirements for specified requirements are known.

- (2) Traceability between requirements ensures that lower-level requirement identifies the higher level requirement from which it is derived. It covers backward traceability and forward traceability.

Correction introduced in GS v1.1 START

Lower level requirement, for example a requirement in Entity Functional Specification (EFS) must identify the higher level requirement in System Functional Specification (SFS) on which it is based. It is called "backward traceability".

Backward traceability is recommended in Nokia MN. Details how to implement the backward traceability for requirements created by specification engineers in an Unit or team, shall be defined in the Unit or team local guideline.

Correction introduced in GS v1.1 END

-
- If higher level requirement, for example SFS requirement, identifies lower level requirement(s) created based on the higher level requirement, it is "forward traceability". **Forward traceability is mandatory in Nokia MN if it can be ensured by Requirement Management System (RMS) automation.** E.g. if it is possible to create link between object with SFS requirement and object with EFS requirement. If RMS does not provide such automation and EFS requirement shall be referenced manually from SFS requirement, the forward traceability is recommended but not mandatory.
- In Nokia MN it is allowed also to ensure traceability between requirements on the same level. E.g. EFS requirement #1 is created based on EFS requirement #2. For this kind of requirements traceability, the same rules as for forward traceability shall be applied (see above).
- (3) Post-RS traceability ensures requirement implementation or requirement test cases can refer to the requirement. It is applicable when each requirement have a unique id (e.g. LTE_OPR_REQ_1234 in DOORS). The id is also needed to realize traceability between requirements

Requirements labeling for traceability purpose

To ensure traceability between requirements and Post-RS traceability each requirement has to have its own unique and persistent id (label). It is valid both for basic requirements and SW robustness requirements (see [Complete \(see page 43\)](#) for "basic requirement" and "SW robustness requirement" definition).

It may be:

- either id of object or artifact in RMS (Requirements Management System) OR

- textual id written manually. E.g. if (1) many requirements are stored in the same RMS object or artifact each of them requires manually assigned id OR (2) RMS doesn't use objects or artifacts to store requirements (e.g. svn and rst files are used to store BOAM PFS). See examples in [Atomic or Molecular](#) (see page 31).

Nokia MN teams shall decide in internal guideline if textual id is allowed or each single requirement shall be stored in single RMS object or artifact.

To allow requirements effective traceability recommended option is to use RMS id, not manually written textual id.

- The golden standard does not enforce any specific way to implement requirements labeling.
- Sometimes the labeling style is imposed by tool (RMS) used to document requirements.
- The golden standard does not define what RMS capabilities shall be used for requirements traceability; if textual references with the requirement id (label) or another capability like links between requirements. However to ensure effective requirements traceability RMS inbuilt capabilities are recommended. e.g. links, not traceability executed based on manually written requirements references.
- If you want to know more on possible labeling styles you can find it for example in [Wiegers and Beatty] in chapter "Labeling requirements".

Note

Be careful with links between requirements objects or artifacts management.

- If a requirement object or artifact contains requirement req-x and another object or artifact contains requirement req-y, create a link between the requirement objects or artifacts - only if req-x is created based on req-y. Otherwise the link is not needed.
- If you remove (delete) object or artifact with requirement, delete links created to/from the deleted requirement object or artifact.
- If a group of requirements stored in a module of RMS is moved to another module, links to/from the requirements shall be updated correspondingly.

The rules above shall be also applicable for textual references to requirements.

Labeling of requirements stored in a table

If requirements are stored in a table, each of them has to have its own id. The golden standard does not force to use single way of requirements in the tables labeling in whole MN.

Each team has to decided on SINGLE way of requirements labeling in tables and consequently use the single way in all requirements created by them.

Options of requirements labeling.

Option 1 - requirement id specified in the table

Requirement OM_GEN_SFS_FM_1234

When two alarms are active in BTS at the same time, then BTS shall filter them out based on following rules:

Requirement ID	Alarm#1 severity	Alarm#1 alarming object	Alarm#2 severity	Alarm#2 alarming object	Alarm displayed in NMS
OM_GEN_SFS_FM_1234.1	minor	X	minor	X	Alarm that was raised as first
OM_GEN_SFS_FM_1234.2	major	X	minor	X	Alarm#1
OM_GEN_SFS_FM_1234.3	minor	X	major	X	Alarm#2

Example of reference: "See **OM_GEN_SFS_FM_1234.3**".

Option 2 - requirement number suffix specified in the table

OM_GEN_SFS_FM_1234 When two alarms are active in BTS at the same time BTS shall filter them out based on following rules:

Requirement ID Suffix	Alarm #1 severity	Alarm#1 alarming object	Alarm #2 severity	Alarm#2 alarming object	Alarm displayed in NMS	Feature ID
---------------------------------	-------------------	-------------------------	-------------------	-------------------------	------------------------	------------

1	minor	X	minor	X	Alarm that was raised as first	CB001234-A
2	major	X	minor	X	Alarm#1	CB001234-A
3	minor	X	major	X	Alarm#2	CB005667-B

Example of reference: "See **OM_GEN_SFS_FM_1234.3**".

Option 3 - requirement natural language suffix specified in the table

OM_GEN_SFS_FM_1234 When two alarms are active in BTS at the same time BTS shall filter them out based on following rules:

Requirement ID Suffix	Alarm#1 severity	Alarm#1 alarming object	Alarm#2 severity	Alarm#2 alarming object	Alarm displayed in NMS
[two minor severity alarms]	minor	X	minor	X	Alarm that was raised as first
[major and minor severity alarms]	major	X	minor	X	Alarm#1
[minor and major severity alarms]	minor	X	major	X	Alarm#2

Example of reference: "See **OM_GEN_SFS_FM1234_[two minor severity alarms]**".

Remember - use case is one of the forms of requirements, it is NOT set of atomic requirements.

See [Chapter 1 - Requirements types and requirements documentation forms](#) (see page 5). The consequence is, you must not refer to a single step of the use case or a range of steps (e.g. Ref: Steps 2-5 of Use Case: Carrier aggregation configuration) of the use case for traceability purpose. You shall refer to whole use case for traceability purpose (e.g. Ref: Use Case: Carrier aggregation configuration).

Good and bad examples for Traceable - single statement requirement

- Good examples

Good example 1 (Pre-RS traceability):

In DOORS module SFS requirement is tagged with feature id (CB006758-A in the example below).

Correction introduced in GS v1.1 START

Based on the feature id the requirement reader can find the feature description and customer requirements based on that the SFS requirement is created. The reader can use the feature id to find the feature description and customer requirements which were used to create the SFS requirement. Similarly, tagging SFS requirements with feature id allows to find all requirements created or updated due to the feature.

Correction introduced in GS v1.1 END

OM_GEN_SFS_SYSM.7714

"Cell disabled" alarm shall be raised by NSP for 5G cells that is not in energySaving state

NSP shall not raise "cell disabled" alarm on 5G cells when the cell is disabled for energy saving purpose, i.e. when NRCELL.operationalState is set "disabled" AND NRCELL.energySavingState is set "energySaving".

Rationale: When cell is disabled for energy saving purpose it is intentional cell switching off and the cell shall not be reported to operator as failed.

RAT Features: CB006758-A Cell switch-off (component carrier) for Energy Efficiency

Good example 2 (Traceability between requirements):

In DOORS module with BOAM EFS requirements, the requirement attribute "Rationale" contains SFS requirement ID, based on what the EFS requirement is created. E.g. Rationale = OM_GEN_SFS_SYSM.7654 in the example below. Additionally the EFS requirement has its own id OAM_REQ_RAT_1234.

OAM_REQ_RAT_1234

When 5G cell is in energy saving state i.e. NRCELL.operationalState is set "disabled" AND NRCELL.energySavingState is set "energySaving".

AND then a critical fault is detected by BTS OAM that makes the cell as failed, then BTS OAM shall keep cell the NRCELL.operationalState set "disabled" but change NRCELL.energySavingState from 'energySaving' to 'notEnergySaving'.

Rationale: Energy saving is feature activated and deactivated by customer. When cell is failed customer is not able do activate or deactivate energy saving. See also OM_GEN_SFS_SYSM.7654

Note

The Rationale attribute usage for traceability purposes is workaround in DOORS as link from EFS requirement and SFS requirement is not allowed due to DOORS performance issue.

If it is available in RMS, the recommended approach is not to mix the requirement rationale (clarification why the requirement sounds like it sounds) with id of higher level requirement.

Note

Another way of traceability implementation is to use links in RMS (e.g. DOORS Next Generation), in this case link from OAM_REQ_RAT_1234 to OM_GEN_SFS_SYSM.7654.

The link would be visible both in EFS requirement object OAM_REQ_RAT_1234 and SFS requirement object OM_GEN_SFS_SYSM.7654. This way the single link ensures backward traceability and forward traceability.

It is recommended way of traceability implementation if available in RMS.

Anyway, even if the link is created, it is recommended to have linked requirement id visible in any of requirement attributes. Link is not visible in requirement snapshot send to review in external tool (e.g. Collaborator).

Good example 3 (Post-RS traceability and traceability between requirements):

Requirement has its own id = OM_GEN_SFS_FM_1234 in the example below.

If the requirement contains table with requirements, each row in the table has its own id. E.g. OM_GEN_SFS_FM_1234.1 in the example below.

OM_GEN_SFS_FM_1234

Alarm filtering rules

When two alarms are active in BTS at the same time, then BTS shall filter them out based on following rules:

Requirement ID	Alarm#1 severity	Alarm#1 alarming object	Alarm#2 severity	Alarm#2 alarming object	Alarm displayed in NMS
OM_GEN_SFS_FM_1234.1	minor	X	minor	X	Alarm that was raised as first
OM_GEN_SFS_FM_1234.2	major	X	minor	X	Alarm#1

OM_GEN_SFS_FM_1234.3	minor	X	major	X	Alarm#2
OM_GEN_SFS_FM_1234.4	major	X	major	X	Alarm#1 and Alarm#2
OM_GEN_SFS_FM_1234.5	any	X	any	Y	Alarm#1 and Alarm#2

Note: List of alarming objects is stored in

OM_GE_SFS_FM_8822.

Good example 4 (traceability between requirements)

Let's assume the SFS requirement is

OM_GEN_SFS_FM_5678

*When feature X is activated (actFeatureX is set "true") then while BTS startup BTS shall calculate parameter_y value based on formula **[parameter_y = a+b]** and send parameter_y value to NetAct*

Based on the OM_GEN_SFS_FM_5678 lower level EFS requirement OAM_REQ_CELL_1234 is created. The specification engineer creates RMS link from the OAM_REQ_CELL_1234 to M_GEN_SFS_FM_5678. The example below is correct as:

- there is reference to SFS requirement from EFS requirement and additionally there is a link created between the requirements
- the formula defined in SFS requirement is not copied to the EFS requirement (see also [Modifiable](#) (see page 60))
- it is clearly stated what exactly the requirement actor shall do (calculate parameter_y).

OAM_REQ_CELL_1234

*When feature X is activated (actFeatureX is set "true") and Component_B reports RMOD operational state is Enabled to BTS_OAM then BTS OAM shall calculate parameter_y based on formula defined in **OM_GEN_SFS_FM_5678** and send parameter_y value to NetAct.*

- Bad examples

Bad example 1 (Pre-RS traceability):

In DOORS module SFS requirement is not tagged with feature id and the requirement rationale is not specified.

OM_GEN_SFS_SYSM.7714

"Cell disabled" alarm shall be raised by NSP for 5G cells that is not in energySaving state.

NSP shall not raise "cell disabled" alarm on 5G cells when the cell is disabled for energy saving purpose, i.e. when NRCELL.operationalState is set "disabled" AND NRCELL.energySavingState is set "energySaving".

Rationale:**RAT Features:**

Bad example 2 (Traceability between requirements):

In DOORS module with BOAM EFS requirements, the requirement attribute "Rationale" is empty. Additionally link from the EFS requirement to SFS requirement is not created. The origin of the EFS requirement, i.e. the SFS requirement, is unknown.

OAM_REQ_RAT_1234

When 5G cell is in energy saving state i.e. NRCELL.operationalState is set "disabled" AND NRCELL.energySavingState is set "energySaving".

AND next a critical fault is detected by BTS OAM that makes the cell as failed, then BTS OAM shall keep cell the NRCELL.operationalState set "disabled" but change NRCELL.energySavingState from 'energySaving' to 'notEnergySaving'.

Rationale:

Bad example 3 (Post-RS traceability and traceability between requirements):

The requirement contains table with requirements but rows in the table don't have their own id.

OM_GEN_SFS_FM_1234**Alarm filtering rules**

When two alarms are active in BTS at the same time, then BTS shall filter them out based on following rules:

Alarm#1 severity	Alarm#1 alarming object	Alarm#2 severity	Alarm#2 alarming object	Alarm displayed in NMS
minor	X	minor	X	Alarm that was raised as first
major	X	minor	X	Alarm#1
minor	X	major	X	Alarm#2

major	X	major	X	Alarm#1 and Alarm#2
any	X	any	Y	Alarm#1 and Alarm#2

Bad example 4 (traceability between requirements)

Let's assume the SFS requirement is

OM_GEN_SFS_FM_5678

*When feature X is activated (actFeatureX is set "true") then while BTS startup BTS shall calculate parameter_y value based on formula **[parameter_y = a+b]** and send parameter_y value to NetAct*

Based on the OM_GEN_SFS_FM_5678 lower level EFS requirement OAM_REQ_CELL_1234 is created.. Specification engineer doesn't create RMS link from OAM_REQ_CELL_1234 to M_GEN_SFS_FM_5678.

The example below is bad as:

- the formula is duplicated in the EFS requirement (see also [Modifiable \(see page 60\)](#))
- there is no reference to the SFS source requirement in the EFS requirement
- there is no link between EFS and SFS requirements, neither backward nor forward traceability is ensured.

OAM_REQ_CELL_1234

*BTS OAM shall calculate parameter_y based on formula **[parameter_x = a+b]** when RU reports a and b values to BTS OAM.*

The next example below is also bad as it is not clearly stated what the requirement actor shall do (shall calculate parameter_x) even if contains reference to higher level SFS requirement.

OAM_REQ_CELL_1234

*BTS OAM shall follow **OM_GEN_SFS_FM_5678** when RU reports a and b values to BTS OAM.*

Good and bad examples for Traceable - use case

- Good examples

Good example 1:

In DOORS module with SFS scenarios, the requirement attribute "RAT Features" (CB003434-B in the example below) contains feature id based on which use case was created. The requirement user can use the feature id to check the feature description in Focal Point and find corresponding user requirements.

OM_GEN_SFS_SYSM.2345**Use Case: RTWP fault reporting.****Purpose:**

To report alarm with fault “RTWP level too low” based on RTWP measurements if RTWP level is too low.

Actors:

BTS, NMS, Operator

(...)[Next parts of the Use Case here, not presented in order to limit the example length.]

RAT Feature: CB003434-B

Good example 2: (Traceability between requirements):

BOAM PFS (Plane Functional Specification) document contains header table. The header table shows based on what the BOAM EFS requirement the BOAM PFS use case in is created.

Additionally, if technically doable, there may be link created behind the BOAM EFS requirement id text. When clicking the link, the use case reader is forwarded to RMS (e.g. DOORS) with the BOAM EFS requirement content.

Note: not whole use case is presented below to limit the example length.

UC: IQ Power Scaling for 5G Cells on eCPRI RU ([CellMgmt_5gIqPowerScalingForECPRI](#))

Features Ref.:	cOAM - CNI-55626, 5GC000399-S, CNI-58218, 5GC002519-T, 5GC001053-A, CB007460-5G-A, 5GC001086-A-B1, CB008205-SR-B-B12
Req Ref.:	cOAM - OAM_REQ_RAT_1021, OAM_REQ_RAT_1022
HW unit:	cOAM - A-SM-5_5G, A-SM-6
PIT testable:	Yes
Document Owner(s):	cOAM - I_MN_RAN_BOAM_RD_SERVICEABILITY_ARCH@internal.nsn.com
Deployment:	cOAM

Purpose:

To describe the way OAM retrieves IQ power scaling factors (i.e. ulGainCorrection for PRACH and PUXCH) for each UL subcell of 5G cell on eCPRI radio connected to 5G capable BBMODs/SMODs taking into account the following parameters:

- NRCELLGRP.scs - value of scs for 5G cell that is on NRCELLGRP.nrrCellList
- NRCELL.prachConfigurationIndex - used to retrieve scs for PRACH
- NRCELL.actBeamforming and NRCELL.numOfSubsectors - used to determine the sectorization type for 5G cell
- NRCELL.cellTechnology and NRCELL.nrrarfcn - needed to distinguish between FR1 and FR2 cells
- NRBTS.ulGainEcprCorrection structure containing ulGainEcprCorrectionSCS120, ulGainEcprCorrectionSCS60, ulGainEcprCorrectionSCS30, ulGainEcprCorrectionSCS15 and ulGainEcprCorrectionSCS1_25
- NRBTS.ulGainEcprCorrectionMode1 structure containing ulGainEcprCorrectionMode1SCS120, ulGainEcprCorrectionMode1SCS60, ulGainEcprCorrectionMode1SCS30, ulGainEcprCorrectionMode1SCS15 and ulGainEcprCorrectionMode1SCS1_25
- CABLINK.iqCompression - value of IQ compression on a given cablink

Selected IQ compression format in UL and DL and ulGainCorrection factors is configured in C/U-Plane during cell setup.

Trigger:

Good example 3 (Post-RS traceability and traceability between requirements):

Requirement has its own id. 'CellMgmt_5gIqPowerScalingForECPRI' in the example below.

Note: not whole use case is presented below to limit the example length.

UC: IQ Power Scaling for 5G Cells on eCPRI RU

CellMgmt_5gIqPowerScalingForECPRI

The requirement unique id

Features Ref.:	cOAM - CNI-55626, 5GC000399-S, CNI-58218, 5GC002519-T, 5GC001053-A, CB007460-5G-A, 5GC001086-A-B1, CB008205-SR-B-B12
Req Ref.:	cOAM - OAM_REQ_RAT_1021, OAM_REQ_RAT_1022
HW unit:	cOAM - A-SM-5_5G, A-SM-6
PIT testable:	Yes
Document Owner(s):	cOAM - I_MN_RAN_BOAM_RD_SERVICEABILITY_ARCH@internal.nsn.com
Deployment:	cOAM

Purpose:

To describe the way OAM retrieves IQ power scaling factors (i.e. ulGainCorrection for PRACH and PUSCH) for each UL subcell of 5G cell on eCPRI radio connected to 5G capable BBMODs/SMODs taking into account the following parameters:

- NRCELLGRP.scs - value of scs for 5G cell that is on NRCELLGRP.nrCellList
- NRCELL.prachConfigurationIndex - used to retrieve scs for PRACH
- NRCELL.actBeamforming and NRCELL.numOfSubsectors - used to determine the sectorization type for 5G cell
- NRCELL.cellTechnology and NRCELL.nrarmfcn - needed to distinguish between FR1 and FR2 cells
- NRBTs.ulGainEcprCorrection structure containing ulGainEcprCorrectionSCS120, ulGainEcprCorrectionSCS60, ulGainEcprCorrectionSCS30, ulGainEcprCorrectionSCS15 and ulGainEcprCorrectionSCS1_25
- NRBTs.ulGainEcprCorrectionMode1 structure containing ulGainEcprCorrectionMode1SCS120, ulGainEcprCorrectionMode1SCS60, ulGainEcprCorrectionMode1SCS30, ulGainEcprCorrectionMode1SCS15 and ulGainEcprCorrectionMode1SCS1_25
- CABLINK.iqCompression - value of IQ compression on a given cablink

Selected IQ compression format in UL and DL and ulGainCorrection factors is configured in C/U-Plane during cell setup.

Trigger:

- Bad examples

Bad example 1 (Pre-RS traceability):

In DOORS module SFS requirement is not tagged with feature id.

Note: not whole use case is presented below to limit the example length.

OM_GEN_SFS_SYSM.2345**Use Case: RTWP fault reporting.****Purpose:**

To report alarm with fault "RTWP level too low" based on RTWP measurements if RTWP level is too low.

Actors:

BTS, NMS, Operator

(...)[Next parts of the Use Case here, not presented in order to limit the example length.]

RAT Feature:

Bad example 2: (Traceability between requirements):

BOAM PFS (Plane Functional Specification) document contains header table. The header table doesn't contain information based on what BOAM EFS requirement the BOAM PFS use case is created.

Note: not whole use case is presented below to limit the example length.

UC: CPID Registration (`Common_CpidRegistration`)

Features Ref.:	5GC001570-CA, CB007406-C
Req Ref.:	
HW unit:	Generic
PIT testable:	Yes

← Missing requirement id based on which the use case is created.

Purpose:

To register static Communication Point Identifier (CPID) to RCP CC&S (SysCom).

Actors:

CUOAM, RCP CC&S (SysCom)

Preconditions:

CUOAM process has been started-up in ACTIVE gNB-CU. RCP CC&S (SysCom service) has been started up and initialized.

3.7 Unambiguous

Definition of the characteristic

- A requirement is UNAMBIGUOUS if it is subject to one and only one interpretation.
- The UNAMBIGUOUS requirement is concisely stated without recourse to technical jargon, acronyms (unless defined elsewhere in the requirements document or in the glossary), or other esoteric verbiage.
- The UNAMBIGUOUS requirement does not contain vague subjects, vague adjectives, vague prepositions, vague verbs or subjective phrases.
- The UNAMBIGUOUS requirement syntax follows [Chapter 2 - Requirements Syntax \(see page 14\)](#) (who is doing what and when)
- The UNAMBIGUOUS requirement does not contain negative statements or compound statements.
- If an abbreviation is used in the requirement and the abbreviation is not commonly known (like LTE or TCP/IP e.g.), the abbreviation has to be either clarified in the glossary in the module where the requirement is stored or has to be clarified in a note added to the requirement.
- If the same abbreviation is used for different objects in a requirements repository (e.g. TDD = time division duplex, TDD = test driven development), recommendation is - each time when the abbreviation is used in the module, it has to explain what the abbreviation means in that specific usage. For example, "TDD (time division duplex)".

- If any step of the requirement written as a use case is optional, it has to be clearly stated - what is the guard condition of the optional step. If many optional steps may happen in the use case and the scenario is written not as UML diagram but it is written as text, it is recommended to prepare separate use case for each of the optional steps. Next it is needed to add references to the use cases with the optional steps from the basic use case.
- If some steps of the requirement written as a use case are executed in parallel, it has to be clearly presented. Recommended form of use case with parallel steps is any of the behavior UML diagram. See <https://www.uml-diagrams.org/uml-25-diagrams.html>.
- If any alternative is needed in the requirement, it has to be clearly stated what is the guard condition of each of the alternative parts. Recommended form of use case with alternative is any of the behavior UML diagram.

Communication helps to reduce requirement ambiguity

You'll never remove all the ambiguity from requirements—that's the nature of human language. Most of the time, reasonable people can draw the right conclusions from even a slightly fuzzy requirement. Getting a little help from your colleagues through reviews will clean up a lot of the worst issues, though. [Wiegers and Beatty](#) (see page 79)

Good and bad examples for Unambiguous - single statement requirement

- Good example 1

All terms used in the example are clarified in Notes. Alternatively they could be defined in separate requirements and references to the requirements would be added to the requirement below.

(For the example purpose assumption is all abbreviations used in the requirement like PRB, BTS or CPU are clarified in glossary created in the requirements repository where the requirement is stored.)

When feature 'slow PRB pooling' is not activated (NRBTS.actSlowPrbPooling is set to "false") in BTS and the CPUs usage on baseband cards in BTS is lower than 50% [Note1], then after the 'slow PRB pooling' activation (NRBTS.actSlowPrbPooling is set to "true"), the BTS shall ensure that the CPUs usage is lower than 70% [Note 2].

Note 1: The condition is fulfilled when the usage is lower than 50% for at least 99% of the time when BTS is powered on, counted each hour.

Note 2: The condition is fulfilled when the usage is lower than 70% for at least 99% of the time when BTS is powered on, counted each hour.

- Good example 2

The requirement is unambiguous as:

- It is clearly stated how to activate **service account synchronization to fronthaul devices feature - SECADM.actSyncToFronthaul is set to "true"**.
- It is clearly stated that requirement is valid for 5G Radio Unit only.
- It is clearly stated that the requirement is valid for 3 links and it is well defined what are the types of the links.
- It is clearly stated for what baseband card the requirement is valid (ABIO). Note, if the requirement attribute is "Hardware type" or similar, the baseband card type shall be added to the attribute, not to the requirement text.
- It is clearly stated what is expected observable result of requirement

When customer activates **service account synchronization to fronthaul devices feature (SECADM.actSyncToFronthaul is set to "true")** in the BTS with 5G Radio Unit connected with 2 Nokia-CPRI links AND 1 eCPRI link to ABIO baseband card in the BTS, then the BTS shall be able to setup calls in all cells configured on the Radio Unit.

The calls setup means User Equipment (UE) attached to the cell can establish call with another UE.

- Bad example 1

When customer activates feature slow PRB pooling, then the system shall not be overloaded.

Reasons why the requirement is incorrect from Unambiguous characteristic perspective.

The following questions cannot be answered unambiguously based on the requirement:

- What is "system"?
- What does it mean "activates feature slow PRB pooling"?
- What does it mean "shall not be overloaded"?
- Additionally negative statement ("shall not") is not recommended.

- Bad example 2

*When customer activates **CB001234**, then BTS shall support mixing 2 Nokia-CPRI links and/or 1 eCPRI link between 5G/4G/3G/2G Radio Unit and baseband card.*

Reasons why the requirement is incorrect from Unambiguous characteristic perspective.

The following questions cannot be answered unambiguously based on the requirement:

- What does it mean CB001234 activation? What parameter is needed to activate the feature
- What does it mean "support". It is ambiguous term. What functionality is hidden behind the "supporting"?
- What does it mean "2 Nokia-CPRI links and/or 1 eCPRI link"? Are the three (2+1) links mandatory here OR 2 links-CPRI OR 1 link eCPRI is enough?
- What does it mean "5G/4G/3G/2G Radio Unit"? Is it Radio Unit that is able to transmit and receive data using ALL of the radio access technologies OR that is able to transmit and receive data using at least one of the radio access technologies OR maybe any combination of them?

Ambiguous terms

Each ambiguous term presented in the table below is prohibited in Nokia MN requirements unless the term is defined in the specification repository what is the meaning of the term.

Example: glossary contains: support definition: *"The word "support" used in the requirements stored in the module means to prepare IQ data according to 3GPP TS 12.345 for configuration defined in the requirement".*

In such case requirement like "5G L1 DL shall support 5G FDD FR1 cell with bandwidth 20MHz." is correct even if it contains the word "support"

If the ambiguous term is not defined, specification engineer shall replace it with unambiguous term based on the hints in the table. If still any of the ambiguous terms is used in requirement sent to review, reviewers shall report defect against it.

The table is taken from [Wieggers and Beatty],

Ambiguous terms	Ways to improve them
acceptable, adequate	Define what conditions are acceptable by and how the system can judge this.
and/or	Specify whether you mean "and," "or," or "any combination of" so the reader doesn't have to guess.
as much as practicable	Don't leave it up to the developers to determine what's practicable. Make it a TBD and set a date for that task.
at least, at a minimum, not more than, not to exceed	Specify the minimum and maximum acceptable values.
best, greatest, most	State what level of achievement is desired and the minimum acceptable level of achievement.
between, from X to Y	Define whether the end points are included in the range.
depends on	Describe the nature of the dependency. Does another system provide input to this system, must other software be installed before your software can run, or does your system rely on another to perform some calculation or provide other service?
efficient	Define how efficiently the system uses resources, how quickly it performs specific operations, or how quickly users can perform certain tasks with the system.
fast, quick, rapid	Specify the minimum acceptable time in which the system performs some action.
flexible, versatile	Describe the ways in which the system must be able to adapt to changing operating conditions, performance, or business needs.
i.e., e.g.	Many people are unclear about which of these means "that is" (i.e., meaning that the full list of items follows) and which means "for example" (e.g., meaning that just some examples follow, one made in your name is okay). No confusing your documentation.
improved, better, faster, superior, higher quality	Quantify how much better or faster constitutes adequate improvement in a specific functional area or quality aspect.
including, including but not limited to, and so on, etc., such as, for instance	List all possible values or functions, not just examples, or refer the reader to the location of the full list. Otherwise, different readers might have different interpretations of what the whole set of items being referred to consists or where the list stops.
in most cases, generally, usually, almost always	Clarify when the stated conditions or scenarios do not apply and what happens then. Describe how either the user or the system can distinguish one case from the other.
match, equals, agree, the same	Define whether a test comparison is case sensitive and whether it means the phrase "contains," "starts with," or "is equal." For real numbers, specify the degree of precision in the comparison.
maximum, maximum, optimum	State the maximum and minimum acceptable values of some parameter.
normally, ideally	Identify abnormal or non-ideal conditions and describe how the system should behave in those situations.
optionally	Clarify whether this means a developer choice, a system choice, or a user choice.
probably, ought to, should	Will it or won't it?
reasonable, when necessary, where appropriate, if possible, as applicable	Explain how either the developer or the user can make this judgment.
robust	Define how the system is to handle exceptions and respond to unexpected operating conditions.
scalable, transparent, graceful	Either use "scalable" or "graceful" means by the user. Translate the user's expectations into specific observable product characteristics.
several, some, many, few, multiple, numerous	State how many, or provide the minimum and maximum bounds of a range.
shouldn't, won't	Ty to use requirements as promises, describing what the system will do.
state-of-the-art	Define what this phrase means to the stakeholder.
sufficient	Specify how much of something constitutes sufficiency.
support, enable	Define exactly what functions the system will perform that constitute "supporting" some capability.
user-friendly, simple, easy	Describe system characteristics that will satisfy the customer's usage needs and usability expectations.

New part added in GS v1.1 START
Additional ambiguous terms list

ambiguous term	way to improve it
legacy	Don't use the word in requirement text. The requirement shall show how the requirement actor shall behave, not behaved in the past ("in legacy"). If you want to clarify what was changed in the requirement you can add a note to the requirement with the clarification or document it in "Change History" attribute. In note or "Change History" you can use the word legacy but specify what "legacy" you mean. E.g. legacy configuration, legacy implementation.

New part added in GS v1.1 END

Good and bad examples for Unambiguous - use case

- Good example

The example describes clearly:

- How RTWP monitoring is activated in BTS (by setting the MRBTS/ BTSSCL.actRxMonitoring to "true").
- Based on which rules, alarm is raised (RTWP level is below threshold MRBTS/ BTSSCL.RTWPThreshold is observed constantly for 30 minutes).
- Based on which rules, cell operational state is calculated (impacted cell(s) operational state is Disabled if the RX failure makes the cell is not able to be operational any longer).
- If RTWP alarm is reported for each antenna separately.

Use Case: RTWP fault reporting.

Purpose:

To report alarm with fault "RTWP level too low" based on RTWP measurements if RTWP level is too low.

Actors:

BTS, NMS, Operator

Preconditions:

1. RX monitoring is activated in BTS
by setting the MRBTS/BTSSCL.actRxMonitoring to "true"
2. BTS started periodical monitoring process of RTWP levels of uplink signals received by RX antennas of radios connected to the BTS.
The monitoring is executed separately for each RX antenna

Trigger:

A hardware problem happens on RX antenna or in any hardware transmitting RX data from antenna in BTS hardware resources. The problem makes received signal power reduction.

Main Flow:

1. BTS detects that RTWP level is
below threshold MRBTS/BTSSCL.RTWPThreshold on any of the RX antennas and the too low value is observed for each 30 seconds measurement sample for 30 minutes.
The too low RTWP value shall be treated as RX critical fault.
2. BTS calculates impact of the too low RTWP RX failure to operational state of cell or cells that use the faulty RX antenna or hardware transmitting data from the RX antenna.
3. BTS raises alarm informing operator about cell(s) failure or degradation due to RX failure and sends the alarm to NMS.

Postconditions:

Operator is informed about RX failure with alarm with fault "RTWP level too low" in NMS. The alarm shows what RX antenna and what cells are impacted by the RX failure.

NMS displays the following values of operational state of impacted cell(s):

- Disabled if the RX failure makes the cell no longer operational or
- Enabled (and availability status Degraded) otherwise.

The rules of operational state calculation is stored in OM_GEN_SFS_SYSM_12345.

Exceptions:

Exc. 1a RTWP measurements are unavailable due to any BTS failure. BTS raises alarm informing operator about RTWP measurement issue. The alarm shows for which RX antenna the measurement is not executed.

Bad example - example doesn't describe clearly:

- How RTWP monitoring is activated in BTS?
- How RTWP monitoring is executed (nothing about the RTWP periodical measurements)?
- Based on what rules alarm is raised?
- Is the "RX failure" the same as "too low RTWP"?
- When the fault shall be reported? Immediately or after any time when RTWP value is too low.
- Based on what rules cell operational state is calculated.
- If RTWP alarm is reported for each antenna separately or not
- What alarm shall be reported when RTWP measurements are unavailable?

Use Case: RTWP fault reporting.**Purpose:**

To report alarm with fault "RTWP level too low" based on RTWP measurements if RTWP level is too low.

Actors:

BTS, NMS, Operator

Preconditions:

RX monitoring is activated in the BTS.

BTS monitors RTWP.

Trigger:

Any hardware problem happens in RX.

Main Flow:

1. BTS detects that RTWP level is too low.
2. BTS calculates impact of RX failure to cells.
3. BTS raises alarm informing operator about cell(s) failure/degradation due to RX failure and sends the alarm to NMS.

Postconditions:

Operator is informed about too low RTWP in "RTWP level too low" alarm.

3.8 Verifiable

Definition of the characteristic

Verifiable

- The requirement is VERIFIABLE if there exists some finite cost-effective process with which a person or machine can check that the requirement is fulfilled.
- VERIFIABLE requirements must use concrete terms and measurable quantities. If a method cannot be devised to determine whether the software meets a particular requirement, then that requirement should be removed or revised.
- Non verifiable requirements include statements such as "works well", "good human interface", "shall usually happen", "is supported". These requirements cannot be verified because it is impossible to define the terms "good", "well", "usually" or what "supported" means.

- The testing team representative shall confirm if the requirement is VERIFIABLE as part of the requirement review process.
- **If a requirement isn't VERIFIABLE, deciding whether it was correctly implemented becomes a matter of opinion, not objective analysis.** [Wieggers and Beatty]
- Requirements that are incomplete, inconsistent, infeasible, or ambiguous are also unverifiable. [Wieggers and Beatty]

(Work in progress for verifiable requirements from FH architecture stream #4 [FH Training for testable requirements.pptx](#)⁷)

Good and bad examples for Verifiable - single statement example

- Good example

Requirement can be verified because it uses concrete terms and measurable quantities.

When baseband card temperature is more than Tx value defined in [REQ:1234 in Nokia AirScale Hardware Climate Control Guideline], then BTS shall switch on secondary fan to cool the card.

Link to the Nokia AirScale Hardware Climate Control Guideline: [\[link here\]](#)

Note

It is not recommended to put hard coded values to requirements. Specification engineer shall add reference to root requirement instead. It protects from the values duplication in root document and the requirement.

If the root requirement does not exist, you need to:

- introduce new parameter for reference purpose.
- define the parameter in the requirement module glossary.
- In the module glossary, define the parameter value with explanation of reason behind selecting that value.

*When baseband card temperature is more than **Tbbmax**, then BTS shall switch on secondary fan to cool the card.*

Glossary: Tbbmax is max temperature where baseband card is fully operational without a cooling system support. Tbbmax is 55 Celsius degrees based on test cases executed in CB001234-C.

⁷ https://nokia.sharepoint.com/:p:/r/sites/RANFrontHaulArchitectureDomain/_layouts/15/Doc.aspx?sourcedoc=%7BAC85B4AF-D770-4FA0-B945-D75B77CB696C%7D&file=FH%20Training%20for%20testable%20requirements.pptx&action=edit&mobileredirect=true

- Bad example 1

Requirement cannot be verified because it does NOT use concrete terms and measurable quantities. What does it mean "too high"?

*When baseband card temperature is **too high**, then BTS OAM shall switch on secondary fan to cool the card.*

Good and bad examples for Verifiable - use case

- Good example

The requirement is verifiable as the requirement postconditions refer to fault definition stored in NIDD and the definition allows to decide if the observed behavior is consistent with the requirement or not.

Use Case: Alarm reporting due to fault detected in secondary core in dual-core configuration

Purpose:

This use case describes the fault management in dual-core configurations at high level - scenario being the fault is detected in secondary core.

Note that at the system level (i.e. considering the BTS a black box), the FM procedures are described in the same way for single-core and dual-core configurations, as there is no difference from this point of view i.e. the operator should always be able to visualize faults and alarms of a BTS in the same way, regardless of the number of cores used in the configuration.

For more details, see **OM_GEN_SFS_FM.2941**.

Actors

Operator, BTS, NMS

Preconditions

- The dual-core BTS is commissioned.
- There is established connection BTS – NMS. Connection to NMS is done through the M-plane IP address located in the primary core.
- There is not an active fault in the BTS

Trigger

- Any fault happens in the secondary core of the BTS. According to NIDD, the Fault Category is set to "Customer".

Note: only faults with category Customer are reflected in alarm reported to NMS

Main flow

1. The BTS detects the fault.
2. The BTS calculates alarm type and alarm severity to be reported to NMS due to the fault.
3. The BTS reports the alarm to NMS

Postconditions

The alarm is visible in NMS for Operator.

The alarm type and severity is consistent with the fault definition in NIDD.

Exceptions

EXC.1a. There is a failure during the fault detection:

- See the general FM use cases under OM_GEN_SFS_FM (OM_GEN_SFS_FM.2941).

EXC.2a. There is a failure during the alarm calculation:

- See the general FM use cases under OM_GEN_SFS_FM (OM_GEN_SFS_FM.2941).

- Bad example

The requirement is not verifiable as there is no clear definition of faults correlation rules and tester is not able to check if the corresponding test result is consistent with the requirement or not.

It is not explained - what is the meaning of "stronger fault " and "weaker fault".

Use Case: Alarm reporting due to fault detected in secondary core in dual-core configuration

Purpose:

This use case describes the fault management in dual-core configurations at a high level- scenario being more than one fault is detected in secondary core.

Note that at the system level (i.e. considering the BTS a black box), the FM procedures are described in the same way for single-core and dual-core configurations, as there is no difference from this point of view i.e. the operator should always be able to visualize faults and alarms of a BTS in the same way, regardless of how many cores is built.

For more details, see **OM_GEN_SFS_FM.2941**.

Actors

Operator, BTS, NMS

Preconditions

- The dual-core BTS is commissioned.
- There is established connection BTS – NMS. Connection to NMS is done through the M-plane IP address located in the primary core.
- There is not an active fault in the BTS

Trigger

- Any fault (fault#1) happens in the secondary core of the BTS. According to NIDD the Fault Category is set to "Customer".

- Then next fault (fault#2) happens in the secondary core of the BTS. According to NIDD the Fault Category is set to "Customer".

Note: only faults with category Customer are reflected in alarm reported to NMS

Main flow

1. The BTS detects the fault#1.
2. The BTS detects the fault#2.
3. **The BTS executes the faults correlation to decide what alarm shall be reported to NMS.**
Stronger fault wins.
4. The BTS calculates alarm type and alarm severity to be reported to NMS due to the stronger fault.
5. The BTS reports alarm due to one of the faults to NMS

Postconditions

The alarm reported due to stronger fault is visible in NMS.
The alarm type and severity is consistent with the fault definition in NIDD.

There is no alarm reported due to weaker fault.

Exceptions

EXC.1a. There is a failure during the fault detection:

- See the general FM use cases under OM_GEN_SFS_FM (OM_GEN_SFS_FM.2941).

EXC.2a. There is a failure during the alarm calculation:

- See the general FM use case under OM_GEN_SFS_FM (OM_GEN_SFS_FM.2941)

4 Chapter 4 - Writing Style

Requirements writing and presentation rules

1. Language, grammar, spelling, punctuation

Reference to Writing Principles removed in GS v1.1 START

- Write requirements in US English as it is stated in Writing Principles valid for Nokia.

See: <https://brand.nokia.com/guidelines/the-basics/how-we-talk/writing-rules>

Comment: Reference removed because Writing Principles contain many rules that are not recommended in technical requirements specification.

Reference to Writing Principles removed in GS v1.1 END

-
-
- Follow other rules defined in the Writing Principles especially for abbreviations, acronyms and bullet points.
- Write requirements in complete sentences using proper grammar, spelling, and punctuation.

2. Active voice

- Write in the active voice to make it clear what entity is taking the action described. Passive voice is never as clear and direct as using the active voice. See also [Active voice versus passive voice](https://confluence.ext.net.nokia.com/display/DOCS/Active+voice+versus+passive+voice)⁸.

⁸ <https://confluence.ext.net.nokia.com/display/DOCS/Active+voice+versus+passive+voice>

New part added in GS v1.1 START

- In the single statement requirement precondition part (after "when") the passive voice is allowed. In the requirement part after "then" it is recommended to use active voice.
- Passive voice after "then" is acceptable only if it is clearly defined who shall execute the action defined after "then".

Example

Good example 1 (active voice after "then", recommended) *When parameter actForceUnitAutoReset is set to "true" and the unit is recovered after its failure, then BTS shall execute the unit autonomous reset.*

Good example 2 (passive voice after "then", acceptable but not recommended) *When parameter actForceUnitAutoReset is set to "true" and the unit is recovered after its failure, then the unit autonomous reset shall be executed by BTS.*

New part added in GS v1.1 END**Example**

The following requirement is written in passive voice:

Upon product upgrade shipment, the serial number will be updated on the contract line.

The phrasing "will be updated" is indicative of passive voice. It denotes the recipient of the action (serial number) but not the performer of the action. That is, this phrasing offers no clue as to who or what updates the serial number. Will the system do that automatically, or is the user expected to update the serial number? Rephrasing this requirement into active voice makes the actor explicit and also clarifies the triggering event:

When Fulfillment confirms that they shipped a product upgrade, then the system shall update the customer's contract with the new product serial number.

[Wiegers and Beatty]

3. Clarity and conciseness

- Keep sentences and paragraphs short and direct. See also [Sentence length](#)⁹

⁹ <https://confluence.ext.net.nokia.com/display/DOCS/Sentence+length>

Example

- Good example (part of use case)

Sentences are short, easy to read and understand.

Main flow:

1. Operator downloads new scf to BTS with parameter actFeatureX set "true".
2. BTS checks the parameter value.
3. BTS activates feature X.
4. BTS sends message FeatureActivationStatusReportAck NE3S message to NMS.

(...)

- Bad example (part of use case)

Sentences are long and complicated, hard to follow.

Main flow:

1. While recommissioning procedure operator downloads new scf file to BTS in order to activate feature X, so the scf contains parameter actFeatureX value set "true".
2. When the new scf file is successfully downloaded to BTS, the BTS initiates the scf content validation procedure in order to check the parameter actFeatureX value and activate the feature X and when observes the parameter value is "true", BTS activates feature X.
3. To confirm the feature X activation procedure successful execution the BTS sends message FeatureActivationStatusReportAck NE3S message to NMS.

(...)

- Write requirements in simple and straightforward language appropriate to the user domain, avoiding jargon. More details you can find in [Plain language](#)¹⁰

New part added in GS v1.1 START

- Avoid wishes or emotionally marked statements in the requirements.

New part added in GS v1.1 END

- In single requirements management system (RMS) object you can store more than single requirement. While the requirements documentation consider how many of them shall be stored in the single RMS object to help the requirements reader to read them easily without scrolling down the page with the RMS object. When the scrolling down is

¹⁰ <https://confluence.ext.net.nokia.com/display/DOCS/Plain+language>

needed, spread the requirements across two or more RMS objects based on any key e.g. duplex mode, hardware type, FR1 vs. FR2 etc.

- **Avoid negative requirements** that describes what the system will not do. How do you implement a don't-do-this requirement? Double and triple negatives are particularly tricky to decipher. Try to rephrase negative requirements into a positive sense that clearly describes the restricting behavior. [Wieggers and Beatty]

Example

- Good example (positive requirement)

The system shall allow the user to activate the contract only if the contract is in balance

- Bad example (negative requirement)

Prevent the user from activating the contract if the contract is **not in balance.**

[Wieggers and Beatty]

- Define specialized terms in a glossary.

Glossary

How to prepare glossary for specification repository?

- Define in glossary any specialized terms that a reader needs to know to understand the requirements, including acronyms and abbreviations.
- Spell out each acronym and provide its definition.
- Note that data definitions belong in the data dictionary, not the glossary.

[Wieggers and Beatty]

- When you use messages, procedures, parameters names, use their official names defined by standardization organization (e.g. 3GPP for procedures), the interface definition document (for messages and their parameters), NIDD (for configuration objects and parameters). Don't use your own abbreviations of the names and avoid using your own variants of the names.

Example

Instead of writing "UE pooling for TDD activation flag is set "true" use activation parameter name and its MOC (Managed Object Class) " UE pooling for TDD activation flag (NRBTS.actUEPoolingTDD) is set "true".

Instead of writing "BTS OAM reads cell parameter message response." use official name of the message "BTS OAM reads CpRtCm_CellParametersResp message".

Instead of writing "EPS bearer context release procedure" use the procedure name standardized by 3GPP "EPS bearer context deactivation procedure".

- When items listed in a sentence or as bullets have a consistent grammatical structure, users can focus on the central idea. (Parallelism, see also [Parallelism](#)¹¹)

Example

- Good example (part of use case)

Main flow:

1. Operator downloads new scf to BTS with parameter actFeatureX set "true".
2. BTS checks the parameter value and- activates feature X.
3. BTS sends message FeatureActivationStatusReportAck NE3S message to NMS.

(...)

- Bad example (part of use case)

Main flow:

1. Operator downloads new scf to BTS with parameter actFeatureX set "true".
2. The parameter actFeatureX validation by BTS.
3. Message FeatureActivationStatusReportAck NE3S sending to NMS.

(...)

Why it is bad example?

¹¹ <https://confluence.ext.net.nokia.com/display/DOCS/Parallelism>

The good example above, using active voice and same structure is the clearest. This sets a template in the mind of the reader.

A does action x

B does action Y

C does action Z

Step 1 in the bad example is written in the active voice, which is good. However, in step 2 and 3 no verb is used and makes the requirement much less clear and open to misunderstanding.

Correction made in GS v1.1 START

- Excluding mathematical formulas, don't use mathematical operators like =, !=, ==, ≠, +, −, ×, ÷, /, ^, <, >, ≤, ≥, >=, <=, &, &&, |, || in requirements, **especially don't use them in logical expressions and conditions**. If you make mistake it may have significant impact on functionality developed based on the requirements. Use full words instead. E.g "is equal" replaces "=", "is equal to or smaller than" instead of "<=", "multiplied by" instead of "x".
- If some or all of the operators are allowed in requirements written in pseudo-code form, will be clarified in Golden Standard v2.0.

Correction made in GS v1.1 END

Example

- Good examples:

Example 1:

*When feature slow PRB pooling is not activated (**NRBTS.actSlowPrbPooling is set to "false"**) in BTS and the CPUs usage on baseband cards in BTS is lower than 50% then after the slow PRB pooling activation (**NRBTS.actSlowPrbPooling is set to "true"**) the BTS shall ensure CPUs usage lower than 70%.*

Note to the Example 1: For the example assumption is there is another atomic requirement that specifies what happens if the CPU usage is equal or higher than 50% before the PRB pooling activation.

Alternatively the requirement can cover the both cases if it is written in tabular form like here:

Example 2:

*After feature slow PRB pooling activation (**NRBTS.actSlowPrbPooling value is changed from "false" to "true"**) BTS shall ensure following CPU usage on baseband cards:*

Id	CPU usage before slow PRB pooling activation	CPU usage after PRB pooling activation
1	lower than 50%	lower than 70%
2	equal or higher than 50%	lower than 75%

Example 3 (part of use case)

(...)

Preconditions:

1. *Cell bandwidth (NRCELL.chBw) is equal to or less than 100MHz.**
2. *The cell operational state is "Enabled"*

(...)

Notes to the example:

- Avoid "up to " Cell bandwidth (NRCELL.chBw) **is up to 100MHz.*** is ambiguous. Shall be the 100MHz taken into account or not?
- Although the requirement is presented as good example it may be still improved. The requirement is valid for bandwidths lower than 100MHz. The question is what are the bandwidths? All bandwidths defined by 3GPP or all bandwidths introduced by Nokia features? This is Nokia requirement, so the response is: the requirement is valid for bandwidths officially introduced by Nokia projects (features).
- Anyway what if Nokia has introduced only some of the 3GPP bandwidths (e.g. only 10MHz and 20MHz) and new Nokia feature introduces new 50MHz bandwidth? In the transition period i.e. during the feature delivery it is not clear if the requirement covers 50MHz also or not. Of course the requirement user can check if the requirement is already tagged with the new feature or not and based on that decide if the new bandwidth 50MHz is already covered by the requirement. To ensure the requirement unambiguity and quick understanding what bandwidths are covered by the requirement it is recommended to list all the bandwidths the requirement is valid for:

Preconditions:

1. *Cell bandwidth (NRCELL.chBw) is one of the following: 10MHz, 20MHz or 50MHz.*
2. *The cell operational state is "Enabled"*

Example 4 - formula is part of the requirement, mathematical operators are allowed:

When LNBTs.prbFactor is equal K in BTS working in TDD duplex mode, then BTS shall assign no more than N PRBs to single UE. The formula of the N calculation:

$$N = LNCEL_TDD.chBw [MHz] / K$$

- Bad examples:

Example 1:

*When feature slow PRB pooling is not activated (**NRBTS.actSlowPrbPooling = "false"**) in BTS and the CPUs usage on baseband cards in BTS is lower than 50%* then after the slow PRB pooling activation (**NRBTS.actSlowPrbPooling = "true"**) the BTS shall ensure CPUs usage lower than 70%*

Example 2 (part of use case)

(...)

Preconditions:

1. *Cell bandwidth (NRCELL.chBw) <= 100MHz.*
2. *The cell operational state = "Enabled"*

(...)

- Use single, agreed in internal guideline of your team, format of formulas. It may be italic font, it may be bold font or another form but only one.

Change made in GS v1.1 START (statements moved from another part of the document + some statements removed + examples updated)

- If you refer a requirement in the requirement text, highlight the referenced requirement id. You can use for example bold fonts for the highlighting.
- Follow local guideline regarding colors, underscores or bold fonts usage in the requirement text to attract reader attention to the requirement specific part. The Golden Standard does not impose any rules of particular part of requirements highlighting, however local guidelines might impose them.

References highlighting

Good example

Assumption: the local guideline of a team imposes highlighting of NIDD parameters names and their values.

*When **NRBTS.actBbPoolingTddFr1Ecpri72e** is set "**false**" and TDD subcell consuming 24 L1UL cell slots is being assigned to L1UL baseband resources, OAM shall place UL subcell only at cell slot index 0, 24, 48, 96, 120, 144.*

- Follow local guideline regarding life cycle of colors, underscores, crossing out in the requirement text. If the local guideline does not define such life cycle avoid using crossing out or underscores. Usually it stays forever, nobody removes them when they are not needed any longer.

Example

Good example

The requirement is changed in feature CB001234-A in release N. Assumption: tagging requirement with feature id to show changes introduced by the feature is defined in local guideline of team that updates the requirement.

*When BTS detects UE type Y in NR TDD cell, then BTS shall reduce downlink transmission speed to the UE to maximum **[before CB001234-A start] 10Mbps [before CB001234-A end] [after CB001234-A start] 20Mbps [after CB001234-A end]**.*

In the release N+1, new changes are introduced due to feature CB005678-A. Assumption: local guideline imposes previous tagging removal.

*When BTS detects UE type Y in NR TDD **[after CB005678-A start] or NR FDD [after CB005678-A end]** cell, then BTS shall reduce downlink transmission speed to the UE to maximum 20Mbps.*

Change made in GS v1.1 END



Part moved to "Complete" chapter in GS v1.1 START

- For requirements with logical expressions in requirement precondition part (after "When....") specify requirement for all possible combinations of the expression attributes. Tabular form of the requirement is recommended here if number of possible behaviors is bigger than two.

Example

- Good example**

Example 1 *When feature X is activated (NRBTS.actXfeature is set "true") and BTS detects UE type Y, then BTS shall reduce downlink transmission speed to the UE to maximum 10Mbps otherwise the BTS shall not reduce downlink transmission speed.* Note: The "otherwise..." part covers all remaining combinations of the requirement precondition parameters. It makes tabular form of the requirement is not needed. The remaining combinations are:

- feature X is not activated and BTS detects UE type Y OR
- feature X is not activated and BTS detects another UE type than Y OR
- feature X is activated and BTS detects another UE type than Y.

If the "otherwise..." part is missing, it is indication for the requirement user that there are other requirements that describes the requirement actor behavior for all the remaining combinations. Alternatively the requirements actor can use single requirement in tabular form to specify the requirement actor for all possible combinations of the precondition parameters. Example 2 *BTS shall use following downlink transmission speed depending on the feature X (NRBTS.actXfeature) activation and detected UE type:*

Id	NRBTS.actXfeature	UE type	DL transmission speed
1	true	Y	maximum 10Mbps
2	true	another than Y	maximum 20Mbps
3	false	Y	maximum 20Mbps
4	false	another than Y	no speed reduction

- Bad example**

When feature X is activated (NRBTS.actXfeature is set "true") and BTS detects UE type Y, then BTS shall reduce downlink transmission speed to the UE to maximum 10Mbps. (Assumption is that other requirements that don't cover remaining combinations of the precondition input parameters don't exist). Why it is bad example? It is not stated what shall happen when:

- feature X is not activated and BTS detects UE type Y OR
- feature X is not activated and BTS detects another UE type than Y OR
- feature X is activated and BTS detects another UE type than Y.

Shall BTS not limit the speed at all? Or maybe shall limit to 20Mbps? Or maybe behave another way?

Part moved to "Complete" chapter in GS v1.1 END

- Don't keep supplementary tags that are needed temporary in the requirements documentation. Remove them when they are not needed any longer. Keeping them makes mess in the requirements documentation and disturbs in reading and understanding them.

Example

- Good example

Release based requirement version for feature N contains ids of features that has introduced a change to the requirement in release N. The ids shall be removed in the requirement version valid for release N+1 and next.



- Bad example

The set of requirement contains a lot of old tags with features ids that are not needed any longer. The group of requirements shall be splitted, each requirement shall be stored in separate repository item (object in DOORS e.g.) and the tags shall be removed.

Any packet dropped outside of the firewall (for example by the `linux` kernel) is not counted as part of this counter.

1a) DESIGN LIMITATION/DESIGN NOTE: Due to design of ASIK – TOP packets are rate limited outside firewall and hence these are not included as part of this counter.

Egress ICMP packets dropped due to rate limit (specified in TT_TTSEC.1155) are not counted (not applicable to 5GRAC)

Rate-limited egress ICMP error messages, generated in response to ingress packets that fail the protocol/expiry validity checks, are counted on `airscale`, due to implementation limitations on `airscale`, (not applicable to 5GRAC)

[5GC000265] Packets are counted per instance of RAL [5GC000265]

[5GC000540-D-C] Packets are counted as whole Classical qNB [5GC000540-D-C]

[RC0916 with PR361498] Packets are counted per instance of TNL, it means per VM and per RAP.

[5GC001024] Packets are counted per instance of TNL, it means per VM and per gNB DU.

[5GC000591] This 5G-RAU level counter shall also include the 5G-RAU core plug-in unit data in case of the split-RAU architecture (`airscale+Airframe`) [5GC000591]

[RP014588/FP001621/PR297037/RP0013075] ~~the number of dropped packets due to ingress rate limiting on the C-Plane interface for secondary core due to architectural limitation~~ shall not be updated and reported [5GC000591]

[RP002103] CB007672-5G-A

The BTS shall support separate packets dropped counter for each rate limiter.

For details on each counters, refer NIDD.

[RP002103] [CB007672-5G-A]

[PR515558]

Non-IP packets (eg., ARP and L2 level service OAM) dropped due to rate limit are not counted in this counter, on FSMR3, ASMR4, ASMR6 and later hardware releases.

That is, non-IP packets dropped due to rate limit are not counted in this counter for HWs used in SBTs. They are only counted on HWs that are purely for 5G only.

[PR515558]

[CB006558]

The BTS shall support a PM counter to count 'the number of dropped packets due to ingress rate limiting on the C-Plane interface for 5G'. For details, refer to NIDD or CB006558 CFAM document (Section 1.3.10 – Impact on Counters, Performance Monitoring and Tracing)

[CB006558]

[CB007128-B] Note: Counter incrementing takes place only for dropped packets from external interfaces. Packets predicted for egress and received from internal interfaces shall not be counted in case the packets are dropped to any reason. Dropping packets due to reception of Broadcast packets from either external interface or other pods doesn't result in an increment of the drop counter.

[CB007467-H-I and CB007128-A] Any reference to VM should be considered as Pods in the context of the feature [CB007467-H-I-E] External firewall support on vCU CNF and CB007128-A in NIDD

Problems due to the massive tagging and keeping :

- It is almost not possible to read the requirement in fluent way.
- It is very difficult to understand the current product functionality required by the requirement.
- It is very difficult to know the changes introduced by each feature/CNI/pronto, are conflicted or not E.g.
 - 5GC001024: Packets are counted per instance of TNL, it means per VM and per gNB DU
 - RC0916 with PR361498: Packets are counted per instance of TNL, it means per VM and per RAP
- For a new feature need to modify the requirement
 - very challenging to make update on this base
 - when modifying based on it, only add more “mess” to it.
 - very difficult to review such requirement
- Requirement is very big, and continuously getting bigger and messy.

Additional problems, not related to tagging:

- Requirement contains multiple requirements, instead each requirement shall be written as atomic requirement.
- Requirement contains details of counter, but as well refers to NIDD for details. Very confusing what shall be described in NIDD, what shall be here.
- Requirement refers to CFAM content. CFAM is a document needed for feature development, not for documenting requirements.

- When you send specification document to review show clearly what has been added, removed or changed in the document. Review tools usually offer option of original version and new version of the document comparison (“diff”). If you don't highlight

changes, the document reviewers can overlook a change introduced by you or report irrelevant comments.

Example

- Good example 1

Diff in Review Board - reviewer can easily find changes in the document sent to review.

Document original version	Document new version
<pre> 342 +20 +20 363 - For ABIO, ABIN, and ASOE if NR8TS.actBbPoolExtensionTddFr1 is set to TRUE see :ref:`DCM_1362` then fronthaul type 364 is eCPRI, or CPRI, otherwise fronthaul type is eCPRI. 365 - For ABIP, or ABIQ fronthaul type is eCPRI, or CPRI. 366 5. When at least one cell nrCellType is configured "16DL" and cell FabioExt property is set to True, then NR8TS.actLargeBb 367 PoolTddFr1 is set to True see :ref:`DCM_1973` . 368 6. For ABIP: NR TDD Supercell [#note13]_ is not mapped to this baseband pool. 369 370 .. note:: 371 - disubcellType mixing or/and ulSubcellType mixing is supported when DCM_1362 and DCM_1079 are enabled :ref:`DCM_1362` and :ref:`DCM_1079` respectively. 372 - If R&D parameter "rdBbPoolSkipChecks" is activated, only condition (1) shall be checked for activating Baseband pooling o 373 f UL PRBs. 374 375 .. _DCM_1362: 376 BaseBand Pooling for DL PRBs Activation: (``DCM_1362``) 377 +20 </pre>	<pre> 341 +20 lines 362 - For ABIO, ABIN, and ASOE 363 is eCPRI, or CPRI, otherwise fronthaul 364 - For ABIP, or ABIQ fronth 365 5. When at least one cell nrCellT 366 PoolTddFr1 is set to True see :ref:`DC 367 6. NR TDD Supercell [#note13]_ is 368 dio that is mapped to a BB board. 369 370 .. note:: 371 - disubcellType mixing or/and ulSu 372 re enabled :ref:`DCM_1362` and :ref:`DC 373 - If R&D parameter "rdBbPoolSkipCh 374 f UL PRBs. 375 - If there is at least one superce 376 cell/non-pooled board from FR1, If BB 377 SSB number (channel group number) of s 378 379 .. _DCM_1362: 380 BaseBand Pooling for DL PRBs Activatio 381 +21 lines </pre>

- Good example 2

Diff in Collaborator - reviewer can easily find changes in the document sent to review.

Document previous version	Document new version
On ABIA card DSP reset may happen to get timing parameters updated. Requirement ID : 5G_RAN_USR_COM_15904 Priority : AC/M Requirement Type : URSC 2.1.1.1.1.10 US #11: 2G and 3G cells handling in CB007437 feature Removed part is highlighted red. This user scenario describes handling of the Nokia CPRI Primary Link and the Nokia CPRI Secondary link interruption between BBU and RU. There are configured 2G or 3G cells on the RU. This functionality is also valid for RF sharing cases when 2G or 3G cells are configured on the RU. Related subfeatures: CB007437-G Actors: SRAN BTS having 2G or 3G cells.	On ABIA card DSP reset may happen to get timing parameters updated. Requirement ID : 5G_RAN_USR_COM_15904 Priority : AC/M Requirement Type : URSC 2.1.1.1.1.10 US #11: 2G and 3G cells handling in CB007437 feature Limited support for 2G and 3G cells is covered only in Dedicated operation mode. Note: In concurrent operation mode functionality of 2G and 3G cells are not guaranteed. 91 This scenario describes handling of the Nokia CPRI Primary Link and the Nokia CPRI Secondary link interruption between BBU and RU. There are configured 2G or 3G cells on the RU. Related subfeatures: CB007437-G Actors: SRAN BTS having 2G or 3G cells. The following picture illustrates the situation before and after CPRI link interruption: New content is highlighted green.

4. Use examples

For complicated requirements use examples to explain expected system behavior. To decide if requirement is complicated and requires example please ask another specification engineer or developer who will implement the requirement if the example is recommended.

Example

Example of requirement with example.

11.1 [eOAM] Rule for splitting of SRS streams between multiple links ([OOI_1792](#))

Features:	SGC001086-C, C3009223-SR-8B-B7
Requirements:	OAM_REQ_RAT_1025, OAM_REQ_RAT_1026, SRAN_RU_SF5_1075

When OAM splits DL data streams between links for LTE/NR TDD mMIMO cell (for LTE: BDLxUL configurations), having also SRS streams to be routed [\[24\]](#), [\[22\]](#) and:

1. the cell is deployed on **Loki** based HW (e.g. ABIO, ABIN, ASOI) or **Thor** based HW (e.g. ABIP, ABIC) and
2. the cell is transmitted from RM via eCPRI protocol

then OAM shall guarantee that the number of SRS streams having assigned the same **Receiver delay** [\[27\]](#) and belonging to given cell which are assigned to each link, is equal to the number of DL data streams of the same cell being carried through the same link.

Note

Example 1:

Assumptions:

1. Cell configuration (NRCELLnCellType) BDLxUL
2. NRCELLnMIMOtxArrayMode is set to 'Full 64TRX Array (4x8x2)'
3. DL data streams decided to be split in 2 links
4. Number of streams per symbol = 8 (acc. to Number of SRS per DL symbol (DCM_635))
5. Number of DL symbols = 64 (TRX) 8 = 8
6. 4 DL data streams have been decided to be carried through link1 and remaining 4 DL data streams through link 2.

SRS splitting example 1

DL symbol (receiverDelay)	localCoaIndex for SRS allocated on link1	localCoaIndex for SRS allocated on link2
0	any 4 from 0-7 range	remaining 4 from 0-7 range
1	any 4 from 8-15 range	remaining 4 from 8-15 range
2	any 4 from 16-23 range	remaining 4 from 16-23 range
3	any 4 from 24-31 range	remaining 4 from 24-31 range
4	any 4 from 32-39 range	remaining 4 from 32-39 range
5	any 4 from 40-47 range	remaining 4 from 40-47 range
6	any 4 from 48-55 range	remaining 4 from 48-55 range
7	any 4 from 56-63 range	remaining 4 from 56-63 range

5. Requirement representation

Change made in GS v1.1 START [Moved up to another paragraph of the chapter - look for "Follow local guideline regarding life cycle of colors, underscores, crossing out in the requirement text(...) above".]

- Don't use underscores and and crossing out in requirement content. Usually it stays forever, nobody removes them when they are not needed any longer.

[Example removed. New good examples created in another paragraph of the chapter - look for "Follow local guideline regarding life cycle of colors, underscores, crossing out in the requirement text(...) above".]

Example

Bad example: *When feature X is activated (NRBTS.actXfeature is set "true") and BTS detects UE type Y in NR FDD cell, then BTS shall reduce downlink and uplink transmission speed to the UE to maximum 20Mbps 10Mbps otherwise the BTS shall not reduce downlink and uplink transmission speed.* This is bad example as some parts of the requirement are crossed out and some are underlined. Exceptionally you can **cross out whole requirement** if it is removed and replaced with new requirement. Don't confuse the case with requirement freezing! The frozen requirement is still valid but for some previous releases. Specification engineer can cross out text of requirement that is not valid for any release any longer.

Change made in GS v1.1 END

- If requirement postcondition depends on many input factors, please use tabular form of the requirement instead of complicated and long sentences. See also examples of requirements in tabular form in sub-chapter [Modifiable](#) (see page 60)

Example

- Good example of requirement specified in tabular form.

Requirement ID	Alarm#1 severity	Alarm#1 alarming object	Alarm#2 severity	Alarm#2 alarming object	Alarm displayed in NMS
OM_GEN_SFS_FM_1234.1	minor	X	minor	X	Alarm that was raised as first
OM_GEN_SFS_FM_1234.2	major	X	minor	X	Alarm#1
OM_GEN_SFS_FM_1234.3	minor	X	major	X	Alarm#2
OM_GEN_SFS_FM_1234.4	major	X	major	X	Alarm#1 and Alarm#2
OM_GEN_SFS_FM_1234.5	any	X	any	Y	Alarm#1 and Alarm#2

Why it is good example?

- Each row of the table contains atomic requirement
- Each requirement has its own id (see column "Requirement ID")
- There are not any empty cell in the table.
- Bad example of requirement written in tabular form.

Alarm#1 severity	Alarm#1 alarming object	Alarm#2 severity	Alarm#2 alarming object	Alarm displayed in NMS
minor or major	X	minor	X	Alarm raised unless has s
minor	X	major	X	Alarm
major	X	major	X	Alarm Alarm
	X		Y	Alarm Alarm

Why it is bad example?

- First row (yellow) does not contain atomic requirement.
- There are not requirements ids for each requirement.
- There are empty cell in the table (blue). It is not clear how to understand the condition in empty field. As "any"? As "N/A" (not applicable) Or maybe it is missing value?

6. Use inclusive terms only

Inclusive language or Equity Language is a language style that avoids expressions that its proponents perceive as expressing or implying ideas that are sexist, racist, or otherwise biased, prejudiced, or insulting to particular group(s) of people; and instead uses language intended to avoid offense and fulfill the ideals of egalitarianism.. [Wikipedia, Inclusive Language]

Avoid non-inclusive terms in MN requirements specification. If you find non-inclusive term in legacy requirement, replace it with alternative inclusive term.

Examples

Non-inclusive terms and alternatives:

Non-inclusive term	Examples of alternative terms
master (when used in "master / slave" context)	primary, controller, main
slave	secondary, standby
white list, whitelist, white-list	allow list, accept list
black list, blacklist, black-list	block list, drop list, forbidden list
grey list, greylis, grey-list (a term which has been used in conjunction with white list and black list) should be replaced with e.g. track list, inspect list.	

Source: [3GPP, inclusivity]¹²

More examples you can find here: <https://news.uoguelph.ca/guides/style-guide/inclusive-language/inclusive-it-language-replacement-terms/>

¹² <https://www.3gpp.org/news-events/3gpp-news/inclusivity>

- If requirement written as use case contains optional steps, alternatives, parallel actions, use rather diagram (e.g. UML diagram) than textual form.

Final hint

For each piece of information in the requirements set, ask yourself, “What would the reader do with this information?” If you aren’t certain that some stakeholder would find that information valuable, perhaps you don’t need it. [Wiegers and Beatty]

5 Chapter 5 - Notes Usage

Requirements vs Notes

A 'Note' must not be confused with requirements.

Requirement definition you can find in [Chapter 1 - Requirements types and requirements documentation forms](#) (see page 5)

A 'Note' is supplementary information that is only used for reasons mentioned below.

Note's correct usage:

- Note must be associated with a requirement or group of requirements. In other words, notes shall exist only jointly with the commented requirement statement in the same requirement object unless it is stated another way in the points below.
- Note shall contain any one of below aspects, or multiple of them:

Removed in GS v1.1. START

- ~~Supplementary explanation needed to make the requirement unambiguous.~~
- ~~Definition of any term used in the requirement.~~

~~If the definition is valid for the single requirement only, the note shall be included in the requirement object.~~

~~If the definition is also valid for other requirements, it shall be specified in separate object as it allows to assign specific id (label) to the note with the definition.~~

Removed in GS v1.1. END

- Reference to another requirement or standard document or recommendation document if another requirement or document contains definition of any term document used in the requirement and the term is not defined in the glossary. The reference to another requirement can be also used in requirement content if it does not interfere with the requirement readability. See good example 4 below.
- Supplementary, contextual information that brings wider view of the requirements usage. E.g. Note can provide reference to hardware documentation if requirement contains info on any hardware limitations.
- Requirement rationale if there is no requirement attribute "Rationale".
- Any other information which shall help to understand the requirement. For example, the note can contain the business context of the requirement.

New part added in GS v1.1 START

- MN teams can decide on their own where to document notes in their requirements repository in Requirement Management System (RMS).
 - If in dedicated attribute (e.g. "Notes") (option 1) or
 - In the same requirement object (the same artifact) in which requirement text is stored, under requirement text (option 2). This option is allowed even if object (artifact) with requirement has attribute "Note(s)".
- If option 2 is chosen, specification engineer needs to add word "Note" or "Notes" where the note starts.
Note shall be documented at the bottom of requirements object, not between requirements stored in the same requirement object or in the middle of requirement.
- The note content shall be up to date. In both options, each time when requirement or requirements are changed, specification engineer shall check if note assigned to the requirement is still valid. If not, specification engineer shall update the note.
- If more then single note is assigned to single requirement or to more requirements, the notes shall be labeled with id (e.g. Note 1.... Note 2....) to give the note user possibility to refer to each of them.

New part added in GS v1.1 END

Note's incorrect usage examples:

- Note contains requirement. Resolution: ask yourself if the statement describes expected system behavior, non-functional requirement or restriction. If yes, it is a requirement, and not a Note.
- Note provides information on unsupported functionality. Resolution: The corresponding requirement shall specify clearly what functionality is supported. If anything is not mentioned in the requirement as supported, it is unsupported.
- Note contains info the requirement is not implemented. Resolution: If requirement is not implemented it shall be removed from specification repository (or tagged as Deleted).

Good examples of note usage**Good examples of note usage**

Good example 1 - Note is used to specify reference to definition of a parameter used in the requirement

Note

- For ULPrbPoolingTddFr1 definition see [BaseBand Pooling for UL PRBs Activation: \(DCM_1079\)](#).

Example removed in GS v1.1 START Good example 2 – Requirement and corresponding note with definition of parameter used in the requirement. The definition is valid for the requirement only.

3.3.1. L2 baseband occupancy for TDD NR cell group (DCM_1040) [1]

Features:	RP003279-QBA-a, RP003279-QCA-a, CB008325-5G-BA35-B1, CB008325-5G-CA1-B1, CB009028-5G-F-B2, CB009029-5G-G6-B1, CB009363-SR-A, CB009055-5G-G4-B1
Requirements:	5G_UP_4127, 5G_UP_6391, 5G_UP_7311

When NR cell group is being assigned to L2 baseband resources, OAM shall assume following occupancy of L2 resources:

ID	Release	L2 instance type	NR cell group type	UePoolingTddFr1	CS consumption	Additional information
DCM_1040.1	23R4 onwards	NR_L2RT_SNF_TDD_FR1	GC16	N/A	24	
DCM_1040.2	23R4 onwards	NR_L2RT_SNF_TDD_FR1	GC2/GC4/GC8	OFF	24	
DCM_1040.3	23R4	NR_L2RT_SNF_TDD_FR1	GC2/GC4/GC8	ON	12	
DCM_1040.3	24R1 onwards	NR_L2RT_SNF_TDD_FR1	GC2/GC4/GC8	ON	12	No Supercell [8]
DCM_1040.4	23R4 onwards	NR_L2RT_SNF_TDD_FR1	GC2/4	N/A	24	Supercell [8]

Note

NR Cell Groups are defined as:

- GC2 - NR cell group with TDD cells with nrCellType == 2DL2UL
- GC4 - NR cell group with TDD cells with nrCellType == 4DL4UL or 4DL2UL
- GC8 - NR cell group with TDD cells with nrCellType == 8DL8UL or 8DL4UL
- GC16 - NR cell group with TDD cells with nrCellType == 16DL4UL or 16DL8UL
- GA2 - NR cell group with FDD cells with nrCellType == 2DL2UL or 1DL2UL
- GA4 - NR cell group with FDD cells with nrCellType == 4DL4UL

Example removed in GS v1.1 END

Good example 3 - Note contains supplementary info where additional information of parameter used in the requirement usage can be found.

Note

For more information about the usage of parameters define in the table see [L2 subpool common requirements \(DCM_828\)](#)

Good example 4 - Note contains supplementary info that helps to understand what the context of the requirement is. E.g. here: based on what requirement one of the input parameter's value is chosen. The supplementary info containing references to other requirements is not added to the requirement content (table) as it would interfere with the table readability.

OAM shall create the L2RT instance based on cell slot model during start up according to the SCF configuration and commissioned HW:

ID	L2 Instance type	Number of L2 Pools per L2 Instance	L2 pool list	Duplex	Don
DCM_2259.1	NR_L2RT_vDU_RINLINE2_TDD_FR1	1	NR_L2RT_vDU_RINLINE2_TDD_FR1	TDD	L2R
DCM_2259.2	L2RT_vDU_RINLINE2_FR1	2	NR_L2Pool_vDU_RINLINE2_TDD_FR1 NR_L2Pool_vDU_RINLINE2_TDD_FR1	TDD	L2R

Note

L2 instance type is chosen as described in [L2 deployment to instance type mapping \(DCM_2456\)](#), based on deployment, duplex and cell slot model input parameters retrieval ([DCS_vDUCellSlotModelInputParametersRetrieval](#)).

When reference to another requirement doesn't interfere with the requirement readability, the reference shall be added to the requirement content like here:

OAM_REQ
_RAT_484

BTS shall set MRBTS/INBTS/INBTS_FDD/LAA.lteUtuRegion parameter correctly based on **OM_LTE_SFS_RNL.21469** when at least one LAA object in BTS configuration exists.

Good example 5 - Note contains root cause (rationale) of the requirement

OM_GEN _SFS_S YSM.838 1	BTS shall check connection to LSS server before log collection When destination is set to LSS server, after snapshot is triggered, BTS shall freeze all the files registered to snapshot list and check connection to LSS server before log collection. - If LSS server is reachable, BTS shall continue snapshot collection. - If LSS server isn't reachable, snapshot collection is cancelled. Rationale To avoid data in core dump data lost due to transfer fail caused by connection problem.
----------------------------------	---

Good example 6 - Note contains rationale of the requirement.

4.3. Change in DeploymentId after recommissioning (DCM_1460)

When no cells are deployed on given BB module, in case of recommissioning, OAM:

- shall not change the DeploymentId. (DCM_1460.1)
- shall change the DeploymentId only for ASIM card if the TNL.trSswCapacity value changed. (DCM_1460.2)

When NRCELL(s) with NRCELL.actSupercell = True deployed on given BB module are removed during recommissioning, OAM:

- shall not change DeploymentId for ABIO from 511 to 507 (DCM_1460.3)
- shall not change DeploymentId for ASOE from 502 to 501 (DCM_1460.4)

Note

Rationale: Full ABIO/ASOE deployments represented respectively by the DeploymentId=511 and 502 can support also lower number of cells than 3, so to reduce unnecessary resets during cell deletion OAM shall keep old DeploymentId.

Bad examples of note usage

Bad examples of note usage

Bad example 1 - Note contains information that the requirement is not implemented.

ID	5G_L3_5G L3 Entity Level and Component Requirements (Gorski, Rafał)
5G_L3_25824	Definition: High priority pSRS segment; Low priority pSRS segment; Input: Raw SRS frequency&transmission resource unit data are composed by frequency domain and transmissionComb domain parameters generated by 5G_L3_9649; #SRSresourceInSegment [5G_L3_25834]; Algorithm: CPRT shall segment SRS resource unit according subband index as below: For each symbol: - Find startSubBand as described at 5G_ARG_ALG_4831; - Push total of #SRSresourceInSegment SRS resource units into high priority SRS segment from startSubBand; Note1 - Push rest of SRS resource units into low priority SRS segment from startSubBand + #SRSresourceInSegment; Output: Two SRS resource segments (Note2); <i>Note1: Depending on the order of some subband with different comb, from subband first then then top comb.</i> <i>Note2: due to limited schedule, number of segments not fully align with requirement.</i>

Bad example 2 - Note provides information on unsupported functionality.

5G_CP_ MM_ 4698	✖ This procedure specifies the handover of the UE between neighbour cells belonging to different DUs. [CB008140-GC] Note: Although the following requirements have been extended to inter-gNB CA, CB008140 does not support intra-gNB inter-DU mobility as it applies to classic gNB only. [End CB008140-GC]
-----------------------	--

Bad example 3 - Note contains requirement

Note In case RU supports calibration port automatic switch: • OAM shall select calibration port which is available and listed on first position in supported calibration ports list (according to BPF). • OAM shall select the same calibration port for all cells configured on one/half of RU antenna panel (defined by LNCEL_TDD.mMimoAntArrayMode).

6 Chapter 6 - Statement of Compliance within the requirement

Introduction

New chapter added in GS v1.1 START

Mobile Network Products delivered by Nokia shall follow protocols, recommendations or requirements published by standard organizations ensuring mobile networks products of various vendors are compatible with each other e.g. organizations gathered under 3GPP banner. Specification engineer of any of Nokia MN architecture elements shall unambiguously document for which elements of the external item (external item = externally issued protocol, recommendation or requirement) Nokia MN architecture element shall be fully compliant and for which partially compliant.

The confirmation that any element is consistent with external item will be called further "Statement of Compliance" (SoC).

To specify correctly SoC specification engineer shall specify:

- external document name and its version the SoC refers to.
- Name of protocol element (e.g. any message parameter name) or recommendation id or requirement id to which the SoC refers to.
- If it is full compliance or partial compliance.
- If Nokia MN architecture element shall be partially compliant with any external item, specification engineer shall clearly document for what part of the external item MN system shall be compliant. MN Unit can decide if information for what part the MN system is compliant shall be stored only in internal Nokia MN requirements or in customer documentation also.
- How system shall behave when unsupported message element or value in the message element comes to the MN system.
- For which software release the SoC is valid.

The Golden Standard recommendation is that the SoCs for specific external item shall be collected by specification engineer in the table added to single statement requirement.

Good example

The requirement below documents system X is partially compliant for message PDUSessionResourceSetupRequest handling. For some elements of the message system X is fully compliant in release 22R3 (green frame), for some elements it is not compliant in release 22R3 (red frame).

System X shall be compliant with PDUSessionResourceSetupRequest message handling of interface Y version A documented in [reference to interface Y specification here] as it is stated in the table:

IE/Group Name	Presence	Range/Type	Description	First Feature	Statement of Compliance
PDUSessionResourceSetupRequest					[22R3] Partially Compliant
AMF-UE-NGAP-ID	M	0..1099511627775 (AMF-UE-NGAP-ID)	5G_CP_IF_NTW_3029	5GC752-A	[22R3] Compliant
RAN-UE-NGAP-ID	M	0..4294967295 (RAN-UE-NGAP-ID)	5G_CP_IF_NTW_3036	5GC752-A	[22R3] Compliant
RANPagingPriority	O	1..256 (RANPagingPriority)	5G_CP_IF_NTW_3124	5GC752-A CB008225-1	[22R3] Not Compliant
NAS-PDU	O	OCTET STRING (NAS-PDU)	5G_CP_IF_NTW_3049	5GC752-A	[22R3] Compliant

If the not compliant IE is sent to the System X, then the System X shall reject the message reception and report minor severity fault 123.

New chapter added in GS v1.1 END

7 Chapter 7 - Document sources, document owner and change history

Bibliography

Books and documents used to prepare the Golden Standard for Requirements Specification

- [IEEE Std 830-1998] IEEE Recommended Practice For Software Requirements Specification - IEEE Std 830-1998 <https://ieeexplore.ieee.org/document/720574>
- [Pohl and Rupp] Klaus Pohl and Chris Rupp "Requirements Engineering Fundamentals", Rocky Nook Inc, Santa Barbara, 2nd Edition 2015
- [Wieggers and Beatty] Karl Wieggers and Joy Beatty "Software Requirements Third Edition", Microsoft Press, Redmond, Washington 2013
- [ISO/IEC25010:2011] International Organization for Standardization: Systems and software engineering – Systems and software Quality Requirements and Evaluation (SQuaRE) – System and software quality models, Geneva 2011
- [Sommerville and Sawyer] Ian Sommerville and Pete Sawyer Requirements Engineering: A Good Practice Guide. Chichester, England: John Wiley & Sons Ltd. 1997
- [Wikipedia, Inclusive Language]: https://en.wikipedia.org/wiki/Inclusive_language
- [3GPP, inclusivity] <https://www.3gpp.org/news-events/3gpp-news/inclusivity>
- [Cohn], Mike Cohn: Succeeding with Agile: Software Development Using Scrum. Upper Saddle River, NJ: Addison-Wesley 2010

Document Owner

The Golden Standard for Requirements Specification owner is Aleksander Rusiecki (aleksander.rusiecki@nokia.com).

The golden standard deployment

The golden standard shall be followed by MN specification engineers for each new requirement created after the golden standard approval and official training on it (planned November 2023).

Regarding legacy requirements the golden standard shall be followed and legacy requirement corrected based on the golden standard when the requirement is updated due to a feature, CNI or pronto.

Remaining legacy requirements clean up to follow the golden standard shall be decided in each MN team separately.

Info Session recording and presentation

Link to info session recording and ppt presentation about the Golden Standard. The info session was organized on 14th December 2023.

https://nokia.sharepoint.com/:f/s/boam_team_site/EgL8XtDuEFdMir3-FQnz7kcB2Z0xA3P-U6VHKZxA7s7Bog?e=eNZF0N

Change History

Change History

version	date	name of the changes author	change description
0.1	 18 Apr 2023	 @Aleksander Rusiecki (Nokia)	Draft version of the golden standard created.
0.2.	 12 May 2023	 @Aleksander Rusiecki (Nokia)	Document under review.
0.3	 04 Oct 2023	 @Aleksander Rusiecki (Nokia)	New version of the document created after 1st round of the review and sent to 2nd round of the review.

version	date	name of the changes author	change description
1.0	 13 Nov 2023	 @Aleksander Rusiecki (Nokia)	First version of the golden standard approved in review. https://collab.ext.net.nokia.com/ui#review:id=885151 The Golden Standard has been approved also at SDT (Specification Decision Team) meeting on 17th November 2023
1.1	 15 Dec 2023	 @Aleksander Rusiecki (Nokia)	Link to info session recording and ppt about the Golden Standard added to Chapter 6.
2.0	 05 Aug 2025	 @Aleksander Rusiecki (Nokia)	New version of the Golden Standard published (version 1.1) based on SDT decision on 1st August 2025 Following Confluence versions of the Golden Standard pages contain version 1.1 approved content. Chapter 1 - Requirements types and requirements documentation forms - version 84 Chapter 2 - Requirements Syntax - version 45 Chapter 3 - Characteristics of good requirements - version 6 • Agreed - version 14 • Atomic or Molecular - version 43 • Complete - version 31 • Consistent - version 22 • Modifiable - version 23 • Traceable - version 32 • Unambiguous - version 33 • Verifiable - version 13 • Writing Style - version 32 • Notes Usage - version 18 Chapter 6 - Statement of Compliance within the Requirement - version 1 Chapter 8 - Open Points - version 18

version	date	name of the changes author	change description

Note, the Change History **refers to all chapters of MN Golden Standard for Requirements Specification**. If any change in any of the golden standard chapters or sub chapters, it shall be reflected in the Change History table with detailed info what chapter or sub chapter was changed, how, and by whom.

This cat has joined the discussion regarding the Golden Standard content.



Using one reviewers keyboard it sent following message.

(Nokia) 31/10/2023 16:01
+66666666666666666666666666666666

The message started lively discussion among other Golden Standard reviewers.

8 Chapter 8 - Open points

Note: The golden standard (approved Version 1.1) does not cover yet all aspects of good requirement documentation. The following points that came up during the reviews of 1.0 and 1.1 versions, require further discussions. Based on the agreements/conclusions of the discussions, the golden standard will be updated in subsequent versions.

1. How to specify algorithms?
2. How to specify user stories?
3. Rules of Message Definition Tables (MDT) specification.
4. How manually written requirement id in Requirements Management Systems (RMSes) can be processed by external tools or scripts e.g. for traceability purpose. Example: single RMS object contains n requirements. There are k test plans that cover the some of the n requirements ($k < n$). Currently each test plan refers to the same RMS object id. An automation or script that covers traceability is not able to deduce now that (n-k) requirements are not covered by test plans.
5. In chapter "Writing Style" there is "Excluding mathematical formulas, don't use mathematical operators like =, !=, ==, ≠, +, -, ×". Gregor Kecht comment is "We need to put there "requirement formalization for metacode based on MBSE principles"."
6. Proposal from Gregor Kecht to replace the characteristic name from "verifiable" to "reproducibly testable". "Reproducibly testable" is an attribute which is described in the chapter "testable requirement" here : [Current status of Fronthaul Test Traceability Architecture_V10.pptx](#)¹³
7. CP EFS - We would welcome if we could have second variant of Use Case Template available that by design allows multiple different flows being specified internally. 5G C-plane EFS present but a holistic response a system to a trigger that might produce different outcome. Details can be found in the table for chapter 2 for [5G CP EFS - Golden Standard of Requirement and Specification writing - impact and usage](#).¹⁴

¹³https://nokia.sharepoint.com/:p:/r/sites/RANFrontHaulArchitectureDomain/Shared%20Documents/FH%20Domain%20Streams/S4/Status%20material/Current%20status%20of%20Fronthaul%20Test%20Traceability%20Architecture_V10.pptx?d=wfe6cea8ae4c1420589c4563666e2c35b&csf=1&web=1&e=I9hTpR&nav=eyJzSWQjOjlxNDU3MDY4MjEsImNJCi6MzI1ODAwMTgzNn0

¹⁴<https://confluence.ext.net.nokia.com/pages/viewpage.action?pagelId=1724642755>

8. CP SFS/EFS - how to specify rd parameters? we have a guideline in C-plane for that [R&D Parameter Specification and Removal \(L2/L3\)](#)¹⁵
9. 17. CP EFS for the "unambiguous" that part is having too modest content for such a huge problem. its content shouldbe extended to cover "don't and dos" - catalogue of good bad practices that increase/decrease ambiguity - Golden Standard of specification v.2
10. Rules of pseudo code usage.
11. Traceability - further changes based on Junru team decisions.
12. Maybe we shall discuss what shall be the structure of document (e.g. module in DOORS), templates of documents

¹⁵ <https://confluence.ext.net.nokia.com/pages/viewpage.action?pageId=1265690714>